

Relatório de Auditoria de Segurança — Zyllen Gestão

14 de setembro de 2026

12 achados • 7 altos • 5 médios

198 handlers • 20 controllers • 15 verificações isoladas

ESCOPO

API NestJS/Prisma, frontend Next.js/React, schemas compartilhados, configuração/deploy, documentação, histórico Git local e bundle frontend disponível.

NOTA METODOLÓGICA

As cinco categorias foram adaptadas ao isolamento manual por empresa/dono, aos guards RBAC, aos IDs das rotas NestJS, aos segredos de bootstrap/deploy e aos sinks HTML da aplicação. RLS real e configuração de produção não foram consultados.

Referência: f3b2727190adbf96c9d35dc57b927e79f5c85275 • Leitura e testes sintéticos. Nenhuma conexão ao banco compartilhado; nenhuma alteração ou exploração em produção.

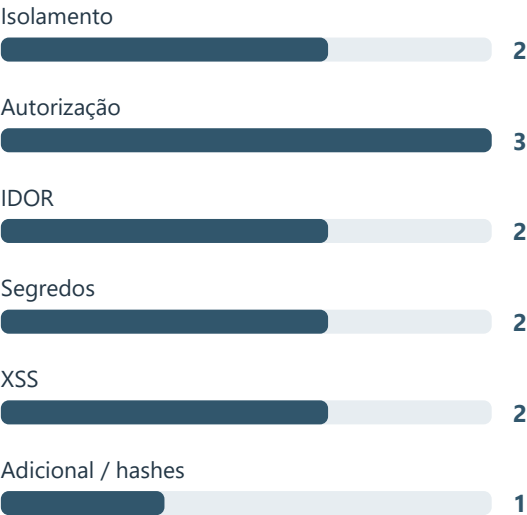
Resumo executivo

As fronteiras de autorização existem, mas o ingresso na empresa e algumas rotas alternativas permitem contorná-las. Os riscos centrais são acesso cruzado a OS, exposição de mídias, alteração de patrimônio e XSS persistente.

Achados por severidade



Achados por categoria



Contagem sem duplicidade: 11 achados nas cinco categorias solicitadas e 1 adicional de exposição de hashes. Um achado pode ter vários arquivos/rotas; A03 agrupa sete handlers públicos mais /uploads. Não há achado crítico, baixo ou informativo contabilizado. Severidade contextual, sem atribuição de CVSS.

Cobertura	Resultado
Handlers / controllers	198 / 20 — inventário completo, sem amostragem de rotas
Histórico local	133 commits; 744 blobs textuais; 30 objetos binários excluídos
Bundle disponível	56 arquivos JS/map; 2.223.711 bytes; Build ID F20QlledV_e2pGVtU-eki
Validações	13 testes offline + 2 reproduções em Chrome isolado
Produção	Nenhuma conexão ao banco ou chamada à aplicação de produção

Stack e método

Camada	Detectado
Linguagem / workspace	TypeScript strict; monorepo pnpm; pacote compartilhado Zod
Backend / dados	NestJS 10; Prisma 6; PostgreSQL no Supabase; conexão de servidor
Autenticação	Passport-JWT; access/refresh; três tipos: internal, external, contractor; RBAC screen.action
Frontend	Next.js 16.1.6 App Router; React 19.2.3; três portais; localStorage para tokens
Deploy encontrado	Dockerfiles API/web; docker-compose.yml; GitHub Actions deploy.yml; ecosystem.config.js; Vercel configs. Documentação indica API manual Windows/Cloudflare e frontend Vercel.
Não aplicável / não encontrado	Não há Helm/Terraform no escopo localizado; nem backend de e-mail/markdown com HTML de usuário identificado.

1. Banco sem tranca

Prisma usa conexão de servidor. Isolamento observado é manual: companyId para OS/acompanhamentos de clientes, externalUserId para seus chamados e openedByContractorId para OS de terceirizados. Não há middleware de tenant no PrismaService. Operação interna é global Skyline, protegida por RBAC; não foi tratada como multi-tenant por funcionário. Políticas RLS reais do Supabase não foram consultadas e sua ausência não foi presumida.

2. Permissão no navegador

Cruzamento de hasPermission, papéis nomeados, botões condicionais e bloqueios de estado com JwtAuthGuard, PermissionsGuard, RequirePermission e verificações nos serviços. As exceções encontradas são rotas alternativas, bloqueio de assinatura incompleto e privilégio de fechamento não aplicado.

3. IDOR

Inventário AST de todos os 198 handlers em 20 controllers. Cada handler foi revisado por audiência, origem do identificador, filtro do recurso pai e política aplicável. Inclui IDs no corpo/query, listagens, totais, históricos, estatísticas e entregas de arquivos; /uploads é superfície extra, fora da contagem de handlers.

4. Chaves expostas

Varredura heurística com revisão dos matches em 222 arquivos textuais versionados, 133 commits locais/744 blobs textuais alcançáveis e 56 arquivos JS/map do bundle local (2.223.711 bytes). Configs, scripts, docs e CI incluídos. Valores sensíveis não são impressos. Literais de seed e JWT são achados condicionais, não confirmação de configuração em produção.

5. XSS

Busca de sinks DOM/HTML, URLs dinâmicas, markdown, eval e templates no backend. Há três pontos de document.write: os geradores de OS/acompanhamento escapam dados; o de etiquetas tem injeção confirmada. Uploads locais fornecem o segundo XSS. Não há biblioteca dedicada de sanitização encontrada nos manifests/código da aplicação; escape contextual manual existe nos dois geradores. Não foram encontrados templates de e-mail ou renderizador markdown/HTML backend alimentados por usuário: esse subitem não se aplica ao código localizado.

Limites da conclusão

Auditoria estática com reproduções locais dirigidas; não é pentest remoto nem certificação de segurança. Todas as rotas foram percorridas, mas a execução dinâmica ficou nos 15 casos registrados. O banco compartilhado, RLS efetivo, valores de .env, arquivos de clientes, backups, logs operacionais e segredos reais não foram consultados. Não houve migração, seed, build da aplicação, deploy ou correção de código.

O histórico inclui apenas refs e objetos disponíveis neste clone, sem fetch de branches remotos ou objetos inalcançáveis. A busca de segredos é heurística e pode não reconhecer formatos desconhecidos; os 30 blobs binários não foram inspecionados. O bundle local existente foi examinado sem reconstruí-lo, e não se afirma que corresponda ao bundle publicado ou ao HEAD.

Alertas de segredo revisados: placeholders de .env.example e matches SQL PRIMARY KEY/CSS/framework foram descartados. Credenciais reais de provedores, chaves privadas ou URLs com senha não foram confirmadas entre os matches revisados. Defaults de seed/JWT permanecem achados condicionais A08/A09. Os trechos usam linhas do código auditado; apenas os valores literais de senha e PIN foram redigidos.

Pontos fortes e pontos fracos

RBAC efetivo no servidor

Rotas decoradas recusam usuários externos e internos sem permissão. O bypass Administrador corresponde ao frontend. JWT revalida usuário ativo e papel/empresa no banco; não confia no role enviado pelo navegador. Teste offline confirmou as negativas.

apps/api/src/modules/access/permissions.guard.ts:18-60; apps/api/src/modules/auth/jwt.strategy.ts:26-84

Chamados do cliente limitados ao titular

Os cinco handlers usam identidade autenticada: consultas filtram externalUserId, detalhe e anexação conferem posse; criação fixa o autor. A proteção de posse do chamado não corrige os problemas independentes de entrega/upload de arquivos (A03/A10).

apps/api/src/modules/tickets/client-tickets.controller.ts:46-81, 98-118, 136-157

OS/acompanhamentos filtrados por empresa

Listagem e total recebem companyId do usuário; detalhe e ações autenticadas comparam empresa. Ausência de empresa é rejeitada. Testes confirmaram empresa alheia negada. Ressalvas: entrada indevida no tenant (A01/A02), assinatura (A06) e arquivos públicos (A03).

apps/api/src/modules/maintenance/client-maintenance.controller.ts:37-115;
apps/api/src/modules/followups/client-followups.controller.ts:15-46

Posse das OS do terceirizado

Handlers autenticados verificam tipo contractor e openedByContractorId. Associações subordinadas conferem recurso pai. Exceções explicitadas: assetId na criação, arquivos públicos e imutabilidade de assinaturas.

apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:47-80, 117-218, 252-354, 385-401

Gestão de usuários, papéis e configurações

Operações administrativas têm guards equivalentes ou mais restritivos que os gates da UI. Agenda separa criar/editar/excluir/gerir instaladores. Não foi encontrado bypass genérico por alterar localStorage ou esconder/reexibir um botão.

apps/api/src/modules/auth/auth.controller.ts:89-159, 219-224; apps/api/src/modules/access/access.controller.ts:33-102;
apps/api/src/modules/schedule/schedule.controller.ts:29-127

Perfil próprio e senha

Atualização de perfil usa req.user.id, permite apenas nome/senha e exige validar a senha atual para substituí-la. roleId e isActive não entram no objeto allowed.

apps/api/src/modules/auth/auth.controller.ts:180-200

Escape nos relatórios HTML de OS e acompanhamento

Texto e atributos dinâmicos passam por escape contextual antes de document.write. URLs de mídia são construídas a partir da API; assinaturas são usadas como imagem, não HTML executável. Não foi confirmado XSS nesses dois geradores.

apps/web/src/lib/os-pdf.ts:55-60, 75-79, 160-184; apps/web/src/lib/followup-pdf.ts:46-48, 66-118, 211-232

Segredos de integração separados do frontend

Chave de serviço Supabase vem do ambiente no servidor. Nenhuma chave privada/API key real foi confirmada nos matches do bundle revisado: 19 alertas eram código de framework/CSS, como `mask-` e `PAGE_SEGMENT_KEY`. Isso não certifica bundles remotos ou históricos não disponíveis.

`apps/api/src/modules/maintenance/maintenance-media-storage.service.ts:22-34`; bundle local `.next/static` (56 arquivos)

Riscos centrais

1. A identidade de empresa pode nascer incorreta, tornando insuficientes filtros posteriores corretos (A01/A02).
2. Arquivos seguem um caminho de autorização diferente dos objetos de negócio (A03).
3. Rotas alternativas e validações incompletas quebram privilégios, posse e imutabilidade (A04–A07).
4. Defaults operacionais e conteúdo persistido ampliam impacto quando as condições descritas se verificam (A08–A12).

Índice de achados por categoria

1. Isolamento de empresa/dono

Severidade	Arquivo:linha	Descrição
ALTA	apps/api/src/modules/registration/registration.service.ts:47-54	A01 — Autocadastro permite ingressar em empresa escolhida pelo solicitante
ALTA	apps/api/src/modules/registration/registration.service.ts:103-114	A02 — Autocadastro apropra OS sem empresa por correspondência parcial de nome

2. Autorização e regras de privilégio

Severidade	Arquivo:linha	Descrição
MÉDIA	apps/api/src/modules/tickets/tickets.service.ts:229-246	A05 — Rota de assumir chamado contorna restrição de transferência de responsável
MÉDIA	apps/api/src/modules/maintenance/maintenance.service.ts:456-461	A06 — Bloqueios de assinatura podem ser contornados por exclusão ou substituição
MÉDIA	apps/api/src/modules/maintenance/maintenance.controller.ts:90-98	A07 — Encerrar OS ignora a permissão específica maintenance.close

3. IDOR e acesso por identificador

Severidade	Arquivo:linha	Descrição
ALTA	apps/api/src/modules/maintenance/maintenance.controller.ts:169-178	A03 — Anexos privados acessíveis por sete rotas públicas e por /uploads
ALTA	apps/api/src/modules/maintenance/maintenance.service.ts:109-124	A04 — Terceirizado altera patrimônio alheio usando assetId ao abrir OS

4. Segredos e credenciais fixas

Severidade	Arquivo:linha	Descrição
ALTA	apps/api/prisma/seed.ts:218-233	A08 — Seed e documentação publicam credenciais fixas de administrador
ALTA	docker-compose.yml:11-14	A09 — Docker fornece segredo JWT público e startup aceita o valor padrão

5. XSS e conteúdo ativo

Severidade	Arquivo:linha	Descrição
MÉDIA	apps/api/src/modules/tickets/client-tickets.controller.ts:17-23	A10 — Upload de HTML disfarçado de imagem executa JavaScript na origem da API
ALTA	apps/web/src/app/dashboard/etiquetas/page.tsx:224-229	A11 — Template de etiqueta injeta script no HTML de impressão do frontend

6. Adicional: exposição de hashes

Severidade	Arquivo:linha	Descrição
MÉDIA	apps/api/src/modules/clients/clients.service.ts:192-207	A12 — Listagem de clientes devolve passwordHash sem necessidade

A01 · Autocadastro permite ingressar em empresa escolhida pelo solicitante

ALTA

Isolamento de empresa/dono

Problema. POST /register/client é público e usa companyId informado no corpo depois de verificar apenas se a empresa existe. A conta nasce ativa. Também aceita projectId sem validar a relação com a empresa. O login passa a emitir uma identidade legítima da empresa escolhida; os filtros posteriores por companyId deixam de distinguir o invasor dos clientes reais.

Explorabilidade. Cadastro público habilitado; atacante conhece o UUID da empresa ou informa nome/CNPJ que coincidem com uma empresa existente. A busca pública de empresas fornece id, nome e CNPJ. Não requer conta prévia, convite, comprovação de vínculo ou aprovação no fluxo encontrado.

Impacto. Leitura das OS e dos acompanhamentos da empresa e ações de assinatura autorizadas a seus clientes. Não foi alegado acesso a chamados pessoais de outros usuários: esses endpoints usam externalUserId.

Verificação. Reprodução offline executou RegistrationService.registerClient com empresa/projeto sintéticos alheios e confirmou que os identificadores foram persistidos, sem isActive=false.

Correção recomendada. Vincular empresas existentes somente por convite de uso único emitido por responsável autorizado ou por aprovação interna. Resolver companyId e projectId no servidor e validar que o projeto pertence à empresa aprovada. Deixar cadastros não aprovados inativos e não executar vinculações de dados durante o cadastro público.

Evidência — arquivo por arquivo

apps/api/src/modules/registration/registration.controller.ts:18-21

Rota pública, somente throttling.

```
18:   @Post('client')
19:   @UseGuards(ThrottlerGuard)
20:   @Throttle({ auth: { ttl: 60000, limit: 30 } })
21:   async registerClient(
```

apps/api/src/modules/registration/registration.service.ts:47-54

Existência não prova vínculo.

```
47:     if (data.companyId) {
48:       company = await this.prisma.company.findUnique({ where: { id: data.companyId } });
49:       if (!company) throw new NotFoundException('Empresa não encontrada');
50:     } else if (data.companyName) {
51:       company = data.companyCnpj
52:       ? await this.prisma.company.findUnique({ where: { cnpj: data.companyCnpj } })
53:       : null;
54:     if (!company) {
```

apps/api/src/modules/registration/registration.service.ts:66-77

Empresa e projeto recebidos são gravados.

```
66:     const user = await this.prisma.externalUser.create({
67:       data: {
68:         name: data.name,
69:         email: data.email,
70:         passwordHash,
71:         phone: data.phone,
72:         city: data.city,
73:         state: data.state,
74:         position: data.position,
```

```
75:         companyId: company?.id || null,  
76:         projectId: data.projectId || null,  
77:     },
```

apps/api/prisma/schema.prisma:65-67

Conta externa ativa por padrão.

```
65:     companyId    String?  
66:     projectId    String? // Projeto vinculado  
67:     isActive     Boolean @default(true)
```

apps/api/src/modules/clients/clients.service.ts:166-175

Descoberta pública dos identificadores.

```
166:     async searchCompanies(query?: string) {  
167:         return this.prisma.company.findMany({  
168:             where: query ? {  
169:                 OR: [  
170:                     { name: { contains: query, mode: 'insensitive' as const } },  
171:                     ...(query.length >= 3 ? [{ cnpj: { contains: query, mode: 'insensitive' as const }  
172:                 ] : []),  
173:             } : {},  
174:             select: { id: true, name: true, cnpj: true },  
175:             orderBy: { name: 'asc' },
```

A02 · Autocadastro apropriada OS sem empresa por correspondência parcial de nome

ALTA

Isolamento de empresa/dono

Problema. Depois de cadastrar um cliente, o serviço busca todas as OS sem companyId e vincula à empresa recém-selecionada/criada aquelas cujo clientName contém o nome da empresa ou é contido por ele. O solicitante controla esse nome; a semelhança textual vira autorização para alterar a posse das OS.

Explorabilidade. Existem OS com companyId nulo e clientName preenchido. Um nome curto aceito pelo schema, por exemplo SA, coincide com SALA PRIVADA na reprodução sintética. Não depende de conhecer o UUID da OS nem de reivindicar uma empresa existente (A01).

Impacto. Transferência indevida de OS legadas para empresa do atacante e posterior acesso via portal. Trata-se de escrita entre inquilinos na operação de cadastro, não apenas de uma listagem sem filtro.

Verificação. Teste do serviço real confirmou updateMany de uma OS sintética SALA PRIVADA para uma empresa sintética SA criada no cadastro público.

Correção recomendada. Remover a reconciliação automática do fluxo público. Executar vinculação legada em operação interna autorizada, com relação explícita e revisão por responsável. Registrar antes/depois na auditoria e preservar os dados existentes.

Evidência — arquivo por arquivo

apps/api/src/modules/registration/registration.service.ts:90-101

```
90:      // Auto-link existing OS whose clientName matches this company (exact or partial)
91:      if (company) {
92:        // Exact match (case-insensitive)
93:        await this.prisma.maintenanceOS.updateMany({
94:          where: { companyId: null, clientName: { equals: company.name, mode: 'insensitive' } },
95:          data: { companyId: company.id },
96:        });
97:        // Partial match: clientName is contained in company name or vice versa
98:        // Load unlinked OS and match in memory to avoid complex DB queries
99:        const unlinked = await this.prisma.maintenanceOS.findMany({
100:          where: { companyId: null, clientName: { not: null } },
101:          select: { id: true, clientName: true },
```

apps/api/src/modules/registration/registration.service.ts:103-114

```
103:      const companyLower = company.name.toLowerCase();
104:      const toLink = unlinked
105:        .filter((os) => {
106:          const nameLower = os.clientName!.toLowerCase().trim();
107:          return companyLower.includes(nameLower) || nameLower.includes(companyLower);
108:        })
109:        .map((os) => os.id);
110:      if (toLink.length > 0) {
111:        await this.prisma.maintenanceOS.updateMany({
112:          where: { id: { in: toLink } },
113:          data: { companyId: company.id },
114:        });
```

packages/shared/src/zod/index.ts:83-86

Nome com apenas dois caracteres é aceito.

```
83:     companyId: z.string().uuid('Company ID inválido').optional(),
84:     companyName: z.string().min(2, 'Nome da empresa é obrigatório').optional(),
85:     companyCnpj: z.string().optional(),
86:     projectId: z.string().uuid('Project ID inválido').optional(),
```

A03 · Anexos privados acessíveis por sete rotas públicas e por /uploads

ALTA

IDOR e acesso por identificador

Problema. Sete handlers de entrega de arquivos usam @Public(). Validam que o anexo pertence ao ID pai da URL, mas não autenticam o leitor nem verificam empresa/dono/permissão. Além disso, ServeStaticModule publica todo o diretório uploads sem JWT. O caminho Supabase também é afetado: um chamador anônimo pode obter uma nova URL assinada por meio do handler público.

Explorabilidade. Atacante possui os IDs opacos da URL ou o nome/caminho do arquivo. A auditoria não demonstrou enumeração de UUIDs. Para /uploads, deve haver arquivo no armazenamento local; para as rotas principais de manutenção, o problema existe também com Supabase Storage.

Impacto. Leitura não autorizada de fotos, vídeos, assinaturas e anexos de OS, acompanhamentos e chamados. A URL externa assinada pode expirar, mas o endpoint público pode renová-la.

Verificação. Todos os sete handlers foram lidos; JwtAuthGuard libera @Public antes da autenticação. O teste de navegador confirmou a disponibilidade de um arquivo sintético no mesmo middleware estático, sem credenciais.

Correção recomendada. Exigir autenticação e política de leitura do recurso pai antes de servir/assinar arquivos. Cobrir interno, cliente e terceirizado sem autorizar apenas pela existência dos IDs. Retirar mídias privadas da raiz estática; se necessário, emitir links temporários vinculados à autorização, sem endpoint público que os renove indefinidamente.

Evidência — arquivo por arquivo

apps/api/src/modules/maintenance/maintenance.controller.ts:169-178

```
169:   @Get('/:id/attachments/:attachmentId/file')
170:   @Public()
171:   async serveFile(
172:     @Param('id') id: string,
173:     @Param('attachmentId') attachmentId: string,
174:     @Res() res: Response,
175:   ) {
176:     const attachments = await this.maintenanceService.findAttachments(id);
177:     const att = attachments.find((a) => a.id === attachmentId);
178:     if (!att) throw new BadRequestException('Anexo não encontrado');
```

apps/api/src/modules/maintenance/maintenance.controller.ts:285-297

```
285:   @Get('/:id/followup-blocks/:blockId/attachments/:attId/file')
286:   @Public()
287:   async serveFollowupFile(
288:     @Param('id') id: string,
289:     @Param('blockId') blockId: string,
290:     @Param('attId') attId: string,
291:     @Res() res: Response,
292:   ) {
293:     const blocks = await this.maintenanceService.findFollowupBlocks(id);
294:     const block = blocks.find((b: any) => b.id === blockId);
295:     if (!block) throw new BadRequestException('Bloco não encontrado');
296:     const att = block.attachments.find((a: any) => a.id === attId);
297:     if (!att) throw new BadRequestException('Anexo não encontrado');
```

apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:220-229

```
220:     @Get('/:id/attachments/:attachmentId/file')
221:     @Public()
222:     async serveFile(
223:         @Param('id') id: string,
224:         @Param('attachmentId') attachmentId: string,
225:         @Res() res: Response,
226:     ) {
227:         const attachments = await this.maintenanceService.findAttachments(id);
228:         const att = attachments.find((a) => a.id === attachmentId);
229:         if (!att) throw new BadRequestException('Anexo não encontrado');
```

apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:356-368

```
356:     @Get('/:id/followup-blocks/:blockId/attachments/:attId/file')
357:     @Public()
358:     async serveFollowupFile(
359:         @Param('id') id: string,
360:         @Param('blockId') blockId: string,
361:         @Param('attId') attId: string,
362:         @Res() res: Response,
363:     ) {
364:         const blocks = await this.maintenanceService.findFollowupBlocks(id);
365:         const block = blocks.find((b: any) => b.id === blockId);
366:         if (!block) throw new BadRequestException('Bloco não encontrado');
367:         const att = block.attachments.find((a: any) => a.id === attId);
368:         if (!att) throw new BadRequestException('Anexo não encontrado');
```

apps/api/src/modules/maintenance/client-maintenance.controller.ts:119-128

```
119:     @Get('/:id/attachments/:attachmentId/file')
120:     @Public()
121:     async serveFile(
122:         @Param('id') id: string,
123:         @Param('attachmentId') attachmentId: string,
124:         @Res() res: Response,
125:     ) {
126:         const attachments = await this.maintenanceService.findAttachments(id);
127:         const att = attachments.find((a) => a.id === attachmentId);
128:         if (!att) throw new BadRequestException('Anexo não encontrado');
```

apps/api/src/modules/maintenance/client-maintenance.controller.ts:145-157

```
145:     @Get('/:id/followup-blocks/:blockId/attachments/:attId/file')
146:     @Public()
147:     async serveFollowupFile(
148:         @Param('id') id: string,
149:         @Param('blockId') blockId: string,
150:         @Param('attId') attId: string,
151:         @Res() res: Response,
152:     ) {
153:         const blocks = await this.maintenanceService.findFollowupBlocks(id);
154:         const block = blocks.find((b: any) => b.id === blockId);
155:         if (!block) throw new BadRequestException('Bloco não encontrado');
156:         const att = block.attachments.find((a: any) => a.id === attId);
157:         if (!att) throw new BadRequestException('Anexo não encontrado');
```

apps/api/src/modules/followups/followups.controller.ts:183-196

```
183:     @Get('/:id/blocks/:blockId/attachments/:attId/file')
184:     @Public()
185:     async serveFile(
186:         @Param('id') id: string,
187:         @Param('blockId') blockId: string,
188:         @Param('attId') attId: string,
189:         @Res() res: Response,
190:     ) {
191:         // Simple serve from disk
192:         const block = await this.followupsService.findById(id);
193:         const blk = block.blocks.find((b) => b.id === blockId);
194:         if (!blk) throw new BadRequestException('Bloco não encontrado');
195:         const att = blk.attachments.find((a) => a.id === attId);
196:         if (!att) throw new BadRequestException('Anexo não encontrado');
```

apps/api/src/app.module.ts:42-46

```
42:         ServeStaticModule.forRoot({
43:             rootPath: join(__dirname, '..', 'uploads'),
44:             serveRoot: '/uploads',
45:             serveStaticOptions: { index: false },
46:         })),
```

apps/api/src/modules/auth/jwt-auth.guard.ts:12-18

```
12:     canActivate(context: ExecutionContext) {
13:         const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
14:             context.getHandler(),
15:             context.getClass(),
16:         ]);
17:         if (isPublic) return true;
18:         return super.canActivate(context);
```

apps/api/src/modules/maintenance/maintenance-media-storage.service.ts:97-103

```
97:     }
98:
99:     const { data, error } = await this.supabaseClient.storage
100:         .from(this.bucket)
101:         .createSignedUrl(objectPath, this.signedUrlExpiresIn);
102:
103:     if (error || !data?.signedUrl) {
```

A04 · Terceirizado altera patrimônio alheio usando assetId ao abrir OS

ALTA

IDOR e acesso por identificador

Problema. A criação de OS no portal terceirizado aceita assetId do corpo. MaintenanceService verifica somente a existência do patrimônio e o associa à OS; em seguida muda seu status para EM_MANUTENCAO. Não há vínculo de autorização entre o terceirizado e esse patrimônio.

Explorabilidade. Conta de terceirizado ativa e UUID de patrimônio existente. O patrimônio não precisa estar relacionado ao solicitante. O cadastro público de terceirizados existe, mas nenhum cadastro real foi feito.

Impacto. Mudança não autorizada de estado operacional e associação de patrimônio de terceiros à OS do atacante. A resposta de criação/detalhe inclui o patrimônio relacionado, expondo informações internas desse recurso.

Verificação. Teste do serviço real com dependências sintéticas confirmou asset.update no UUID alheio e status EM_MANUTENCAO.

Correção recomendada. Não aceitar assetId livre no portal terceirizado. Se o negócio exigir essa associação, consultar uma atribuição explícita autorizada ao chamador antes de criar a OS e atualizar o patrimônio, dentro de uma transação coerente.

Evidência — arquivo por arquivo

apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:83-87

```
83:   async openOS(  
84:     @Request() req: any,  
85:     @Body(new ZodValidationPipe(createMaintenanceSchema)) body: {  
86:       assetId?: string;  
87:       notes?: string;
```

apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:99-103

```
99:   this.assertContractor(req);  
100:   const data = await this.maintenanceService.openOS({  
101:     assetId: body.assetId,  
102:     openedByContractorId: req.user.id,  
103:     notes: body.notes,
```

apps/api/src/modules/maintenance/maintenance.service.ts:59-64

```
59:   // Validate asset if provided  
60:   let asset: any = null;  
61:   if (data.assetId) {  
62:     asset = await this.prisma.asset.findUnique({ where: { id: data.assetId } });  
63:     if (!asset) throw new NotFoundException('Patrimônio não encontrado');  
64:   }
```

apps/api/src/modules/maintenance/maintenance.service.ts:109-124

```
109:         include: {
110:             asset: { include: { sku: { select: { skuCode: true, name: true } } } },
111:             company: { select: { id: true, name: true } },
112:             project: { select: { id: true, name: true } },
113:             openedBy: { select: { name: true } },
114:             openedByContractor: { select: { name: true } },
115:         },
116:     }},
117: );
118:
119: // Update asset status if asset provided
120: if (data.assetId && asset) {
121:     await this.prisma.asset.update({
122:         where: { id: data.assetId },
123:         data: { status: 'EM_MANUTENCAO' },
124:     });
```

A05 · Rota de assumir chamado contorna restrição de transferência de responsável

MÉDIA

Autorização e regras de privilégio

Problema. A UI só mostra Assumir quando não existe responsável e reserva Mover para Administrador/Gestor. O backend restringe reassign, mas assign-with-pin aceita tickets.view e sobrescreve assignedToInternalUserId sem rejeitar um chamado já atribuído. O PIN próprio comprova identidade, não autoriza retirar o chamado de outro técnico.

Explorabilidade. Usuário interno não gestor com tickets.view e PIN válido; chamado ainda não encerrado já atribuído a outra pessoa. Não precisa conhecer o PIN do responsável atual.

Impacto. Apropriação de chamados em execução, alteração de responsabilidade e possibilidade de usar o fluxo de fechamento que passa a reconhecer o atacante como responsável.

Verificação. Reprodução de TicketsService.assignWithPin confirmou a substituição de um responsável sintético pelo chamador não gestor.

Correção recomendada. Permitir autoatribuição apenas se o responsável estiver vazio, em atualização atômica. Qualquer troca de responsável existente deve passar pela política de transferência, inclusive por rotas alternativas. Manter validação de PIN e auditoria.

Evidência — arquivo por arquivo

apps/web/src/app/dashboard/chamados/page.tsx:611-625

```
611:                                     {!d.assignedToInternalUserId && (
612:                                     <Button size="sm" variant="highlight"
        onClick={() => setPinModal({ type: "assign", ticketId: d.id })}>
613:                                     <Shield size={14} className="mr-1" />
        Assumir
614:                                     </Button>
615:                                     )}
616:                                     {!d.assignedToInternalUserId && isAdminOrGestor &&
        (
617:                                     <Button size="sm" variant="outline"
        className="text-white border-[var(--zyllen-border)]"
618:                                     onClick={() => { setDelegateUserId("");
        setPinModal({ type: "delegate", ticketId: d.id }); }}>
619:                                     <User size={14} className="mr-1" />
        Delegar
620:                                     </Button>
621:                                     )}
622:                                     {d.assignedToInternalUserId && isAdminOrGestor &&
        (
623:                                     <Button size="sm" variant="outline"
        className="text-white border-[var(--zyllen-border)]"
624:                                     onClick={() => { setDelegateUserId("");
        setPinModal({ type: "reassign", ticketId: d.id }); }}>
625:                                     <ArrowRightLeft size={14} className="mr-1"
        /> Mover
```

apps/api/src/modules/tickets/tickets.controller.ts:175-184

```
175:     @Put('/:id/assign-with-pin')
176:     @RequirePermission('tickets.view')
177:     async assignWithPin(
178:         @Param('id') id: string,
179:         @Request() req: any,
180:         @Body(new ZodValidationPipe(assignTicketWithPinSchema)) body: { pin: string; assignedToId?:
    string },
181:     ) {
182:         const data = await this.ticketsService.assignWithPin(
183:             id, req.user.id, req.user.role.name, body.pin, body.assignedToId,
184:         );
```

apps/api/src/modules/tickets/tickets.service.ts:229-246

```
229:     async assignWithPin(ticketId: string, userId: string, roleName: string, pin: string,
    assignedToId?: string) {
230:         const ticket = await this.findById(ticketId);
231:         if (ticket.status === TicketStatus.CLOSED) throw new BadRequestException('Chamado encerrado
    não pode ser alterado');
232:
233:         // Only admin/gestor can delegate to another user
234:         const targetId = assignedToId && this.isAdminOrGestor(roleName) ? assignedToId : userId;
235:
236:         const valid = await this.authService.validatePin(userId, pin);
237:         if (!valid) throw new ForbiddenException('PIN inválido');
238:
239:         const user = await this.prisma.internalUser.findUnique({ where: { id: targetId } });
240:         if (!user) throw new NotFoundException('Usuário interno não encontrado');
241:
242:         return this.prisma.ticket.update({
243:             where: { id: ticketId },
244:             data: {
245:                 assignedToInternalUserId: targetId,
246:                 status: TicketStatus.IN_PROGRESS,
```

apps/api/src/modules/tickets/tickets.service.ts:283-287

A rota alternativa de reassign já protege o papel.

```
283:     async reassign(ticketId: string, userId: string, roleName: string, pin: string, newAssignedToId:
    string) {
284:         if (!this.isAdminOrGestor(roleName)) {
285:             throw new ForbiddenException('Apenas administradores e gestores podem mover chamados');
286:         }
287:
```

A06 · Bloqueios de assinatura podem ser contornados por exclusão ou substituição

MÉDIA

Autorização e regras de privilégio

Problema. A UI impede excluir bloco `isLocked` e deixa de oferecer nova assinatura quando `witnessSignature` já existe. A API rejeita editar um bloco bloqueado, mas `removeFollowupBlock` o exclui sem verificar `isLocked`. `clientSignWitness` verifica empresa e status, porém sobrescreve uma assinatura de testemunha preexistente.

Explorabilidade. Exclusão: colaborador com `maintenance.execute` ou terceirizado dono da OS. Substituição: cliente da empresa da OS, ainda não encerrada. Não é acesso a objeto alheio; é bypass das regras de imutabilidade em objetos acessíveis ao chamador.

Impacto. Perda ou troca de evidência de aceite, comprometendo integridade e rastreabilidade das assinaturas confirmadas.

Verificação. Dois testes do serviço real confirmaram exclusão de bloco SIGNATURE `isLocked=true` e sobrescrita de `witnessSignature`. Um teste de controle confirmou que a edição comum do bloco bloqueado é rejeitada.

Correção recomendada. Centralizar a política de imutabilidade e aplicá-la a edição, exclusão, anexos e assinatura. Impedir sobrescrita concorrente de assinatura existente. Correções legítimas devem gerar revisão auditável, sem destruir o aceite original.

Evidência — arquivo por arquivo

apps/web/src/components/os-forms/os-followup-section.tsx:252-256

```
252:     const canDelete = !readOnly && !clientMode && !isLocked;
253:     // Can sign a SIGNATURE block: not readOnly, not locked, (full edit or clientMode)
254:     const canSign = !readOnly && !isLocked && block.type === "SIGNATURE";
255:     // Can confirm/lock: not readOnly, not locked, block is SIGNATURE and has content
256:     const canConfirm = !readOnly && !isLocked && block.type === "SIGNATURE" && !!block.content;
```

apps/web/src/app/portal-cliente/manutencao/page.tsx:186-195

```
186:         {witnessSignature ? (
187:             <div className="space-y-2">
188:                 <div className="rounded-md border border-green-500/30 bg-white p-2">
189:                     <img src={witnessSignature} alt="Assinatura" className="w-full
189: h-24 object-contain" />
190:                 </div>
191:                 <p className="text-xs text-green-400 flex items-center gap-1">
192:                     <Lock size={11} /> Assinatura registrada – não pode ser alterada
193:                 </p>
194:             </div>
195:         ) : !signingWitness ? (
```

apps/api/src/modules/maintenance/maintenance.service.ts:456-461

```
456:     async removeFollowupBlock(osId: string, blockId: string) {
457:         const block = await this.prisma.maintenanceOSFollowupBlock.findFirst({
458:             where: { id: blockId, maintenanceOSId: osId },
459:         });
460:         if (!block) throw new NotFoundException('Bloco não encontrado');
461:         await this.prisma.maintenanceOSFollowupBlock.delete({ where: { id: blockId } });
```

apps/api/src/modules/maintenance/maintenance.service.ts:326-340

```

326:     async clientSignWitness(osId: string, companyId: string, signature: string) {
327:         if (!signature || !signature.startsWith('data:image/')) {
328:             throw new BadRequestException('Assinatura inválida');
329:         }
330:         const os = await this.findById(osId);
331:         if (os.companyId !== companyId) throw new ForbiddenException('OS não pertence à sua empresa');
332:         if (os.status === MaintenanceStatus.CLOSED) throw new BadRequestException('OS já encerrada');
333:
334:         const existingFormData = (os.formData as Record<string, unknown>) || {};
335:         const newFormData = { ...existingFormData, witnessSignature: signature };
336:         validateFormData(newFormData);
337:
338:         return this.prisma.maintenanceOS.update({
339:             where: { id: osId },
340:             data: { formData: newFormData as Prisma.InputJsonValue },

```

apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:306-316

```

306:     @Delete('/:id/followup-blocks/:blockId')
307:     async removeFollowupBlock(
308:         @Request() req: any,
309:         @Param('id') id: string,
310:         @Param('blockId') blockId: string,
311:     ) {
312:         this.assertContractor(req);
313:         const os = await this.maintenanceService.findById(id);
314:         if (os.openedByContractorId !== req.user.id) throw new ForbiddenException('OS não pertence a este terceirizado');
315:         await this.maintenanceService.removeFollowupBlock(id, blockId);
316:         return { message: 'Bloco removido' };

```

apps/api/src/modules/maintenance/client-maintenance.controller.ts:93-102

```

93:     @Put('/:id/witness-signature')
94:     async signWitness(
95:         @Request() req: any,
96:         @Param('id') id: string,
97:         @Body() body: { signature: string },
98:     ) {
99:         this.assertCompany(req);
100:         if (!body.signature) throw new BadRequestException('Assinatura é obrigatória');
101:         const data = await this.maintenanceService.clientSignWitness(id, req.user.companyId,
            body.signature);
102:         return { data, message: 'Assinatura salva com sucesso' };

```

apps/api/src/modules/maintenance/maintenance.service.ts:427-433

Controle correto de edição, ausente na exclusão.

```

427:     async updateFollowupBlock(osId: string, blockId: string, data: { content?: string; order?: number
    }) {
428:         const block = await this.prisma.maintenanceOSFollowupBlock.findFirst({
429:             where: { id: blockId, maintenanceOSId: osId },
430:         });
431:         if (!block) throw new NotFoundException('Bloco não encontrado');
432:         if (block.isLocked) throw new ForbiddenException('Este bloco está confirmado e não pode ser
            alterado');
433:         return this.prisma.maintenanceOSFollowupBlock.update({

```

A07 · Encerrar OS ignora a permissão específica maintenance.close

MÉDIA

Autorização e regras de privilégio

Problema. O contrato de permissões define maintenance.execute e maintenance.close separadamente, mas PUT /maintenance/:id/status exige somente execute. O schema aceita CLOSED e o serviço encerra a OS sem consultar close. A revogação de close não restringe essa operação.

Explorabilidade. Papel personalizado com maintenance.execute e sem maintenance.close. Os papéis operacionais padrão do seed concedem as duas em conjunto; a separação só se manifesta quando as permissões são configuradas de forma diferente. O portal terceirizado tem política própria e não foi considerado sujeito ao RBAC interno.

Impacto. Fechamento de OS e efeitos operacionais associados por usuário sem o privilégio específico de encerramento.

Verificação. Teste isolado autorizou somente maintenance.execute no guard e confirmou que MaintenanceService.updateStatus ainda persiste CLOSED.

Correção recomendada. Exigir maintenance.close quando o estado desejado for CLOSED, mantendo a política adequada às outras transições. Fazer a decisão no servidor e testar combinações de permissões, não apenas os papéis padrão.

Evidência — arquivo por arquivo

apps/api/prisma/seed.ts:87-90

```
87:         { screen: 'maintenance', action: 'view' },
88:         { screen: 'maintenance', action: 'open' },
89:         { screen: 'maintenance', action: 'execute' },
90:         { screen: 'maintenance', action: 'close' },
```

apps/api/src/modules/maintenance/maintenance.controller.ts:90-98

```
90:     @Put('/:id/status')
91:     @RequirePermission('maintenance.execute')
92:     async updateStatus(
93:       @Param('id') id: string,
94:       @Request() req: any,
95:       @Body(new ZodValidationPipe(updateMaintenanceStatusSchema)) body: { status: string; notes?:
    string },
96:     ) {
97:       const data = await this.maintenanceService.updateStatus(id, body.status, req.user.id,
    body.notes);
98:       return { data, message: 'OS atualizada' };
```

packages/shared/src/zod/index.ts:424-426

```
424: export const updateMaintenanceStatusSchema = z.object({
425:   status: z.enum(['OPEN', 'IN_PROGRESS', 'CLOSED']),
426:   notes: z.string().optional(),
```

apps/api/src/modules/maintenance/maintenance.service.ts:225-234

```
225:         const prevTag = os.notes?.match(/\[previousStatus:\w+\]/);
226:         updateData.notes = prevTag ? `${prevTag[0]} ${notes}` : notes;
227:     }
228:     if (status === MaintenanceStatus.CLOSED) {
229:         updateData.closedById = userId;
230:         updateData.completedAt = new Date();
231:         // Restore previous asset status from the notes field (only if asset exists)
232:         if (os.assetId) {
233:             const previousStatusMatch = os.notes?.match(/\[previousStatus:(\w+)\]/);
234:             const previousStatus = previousStatusMatch ? previousStatusMatch[1] : 'ATIVO';
```

A08 · Seed e documentação publicam credenciais fixas de administrador

ALTA

Segredos e credenciais fixas

Problema. O seed calcula hashes de senha e PIN literais conhecidos, cria o usuário administrador ativo e não exige substituição no bootstrap. A documentação divulga as mesmas credenciais. Hashing em repouso não protege uma senha cujo valor inicial é público. update: {} conserva uma conta preexistente, mas a criação continua insegura.

Explorabilidade. Seed executado quando a conta padrão não existe e credenciais não trocadas depois. Não foi testado login nem verificado se o administrador de produção conserva esses valores. O risco é de bootstrap/configuração, não uma afirmação de invasão atual.

Impacto. Acesso administrativo integral às funções protegidas pelo bypass Administrador, se a conta ainda aceitar as credenciais padrão.

Verificação. Literais encontrados no código atual e no histórico desde o commit ae56108d26d255f6ed35746eed5d7776707cbeef (10/02/2026), seed.ts:130-131. Valores de senha/PIN foram redigidos neste relatório para não ampliar sua circulação.

Correção recomendada. Remover senha/PIN padrão do seed, logs e documentação. Exigir bootstrap explícito com segredo aleatório de uso único e troca obrigatória. Verificar e rotacionar credenciais eventualmente derivadas desses valores; tratar o histórico como exposição permanente, sem depender de apagar commits.

Evidência — arquivo por arquivo

apps/api/prisma/seed.ts:218-233

```
218:     const adminPasswordHash = await bcrypt.hash('[CREDENCIAL_PADRAO_REDIGIDA]', 10);
219:     const adminPinHash = await bcrypt.hash('[CREDENCIAL_PADRAO_REDIGIDA]', 10);
220:
221:     const adminUser = await prisma.internalUser.upsert({
222:       where: { email: 'admin@zyllen.com' },
223:       update: {},
224:       create: {
225:         name: 'Administrador',
226:         email: 'admin@zyllen.com',
227:         passwordHash: adminPasswordHash,
228:         pin4Hash: adminPinHash,
229:         roleId: adminRole.id,
230:         isActive: true,
231:       },
232:     });
233:     console.log('  ✓ Admin user created:', adminUser.email, '(PIN: [PIN_REDIGIDO])');
```

PROJETO.md:887

```
887: | **Usuário admin** | `admin@zyllen.com` / senha: `[CREDENCIAL_REDIGIDA]` / PIN:
    | `[CREDENCIAL_REDIGIDA]` |
```

A09 · Docker fornece segredo JWT público e startup aceita o valor padrão

ALTA

Segredos e credenciais fixas

Problema. docker-compose.yml define JWT_SECRET com fallback público. O módulo JWT e a estratégia usam getOrThrow, que rejeita ausência, mas não rejeita um valor conhecido/fraco presente. ConfigModule não contém validador que bloqueie esse default.

Explorabilidade. Deployment recebe o fallback do Compose ou copia esse literal para sua configuração. O Compose atual também contém uma DATABASE_URL SQLite desatualizada: não se afirma que ele sobe como está; a exploração depende de corrigir a conexão PostgreSQL e conservar o JWT inseguro. A configuração real da produção não foi inspecionada.

Impacto. Conhecer a chave permite forjar JWTs para IDs de usuários ativos conhecidos. A estratégia reconsulta usuários no banco, portanto inventar um ID ou apenas escrever role=Administrador no token não basta; ainda assim, a assinatura deixa de provar autenticidade.

Verificação. Fallback atual em docker-compose.yml:12. Histórico também contém fallback no código JWT, por exemplo blob 60b0cecf4d99db928ce8394a8542900a821940d1, auth.module.ts:14. O código atual removeu esse fallback local, mas o Compose e a falta de validação permanecem.

Correção recomendada. Exigir JWT_SECRET explicitamente no deployment e validar no startup comprimento/qualidade mínima e uma denylist dos defaults históricos. Rotacionar qualquer chave implantada derivada do literal e invalidar tokens correspondentes de forma planejada.

Evidência — arquivo por arquivo

docker-compose.yml:11-14

```
11:     - DATABASE_URL=file:./dev.db
12:     - JWT_SECRET=${JWT_SECRET:-change-me-in-production}
13:     - API_PORT=3001
14:     - NODE_ENV=production
```

apps/api/src/modules/auth/auth.module.ts:17-20

```
17:         useFactory: (cfg: ConfigService) => ({
18:             secret: cfg.getOrThrow<string>('JWT_SECRET'),
19:             signOptions: { expiresIn: '1d' },
20:         }),
```

apps/api/src/modules/auth/jwt.strategy.ts:18-23

```
18:         ExtractJwt.fromAuthHeaderAsBearerToken(),
19:         (req: Request) => (req.query?.token as string) ?? null,
20:     ]),
21:     ignoreExpiration: false,
22:     secretOrKey: configService.getOrThrow<string>('JWT_SECRET'),
23: });
```

apps/api/src/app.module.ts:26-29

```
26:     ConfigModule.forRoot({
27:         isGlobal: true,
28:         envFilePath: '.env',
29:     }),
```

A10 · Upload de HTML disfarçado de imagem executa JavaScript na origem da API

MÉDIA

XSS e conteúdo ativo

Problema. Uploads locais confiam no MIME informado no multipart e preservam a extensão original do nome. Um arquivo .html com Content-Type image/png passa pelo fileFilter; o servidor estático determina text/html pela extensão. Ao abrir o link diretamente, o navegador executa o HTML persistido.

Explorabilidade. Usuário autenticado com acesso a um fluxo de upload, inclusive cliente em chamado próprio; vítima abre o arquivo como documento. Confirmado no fluxo local de chamados, independentemente de o storage principal de OS usar Supabase. Não executa simplesmente ao aparecer dentro de uma tag img.

Impacto. XSS persistente na origem da API e conteúdo enganoso servido pelo domínio confiável. A API e o frontend estão em origens distintas: esse vetor, sozinho, não lê o localStorage do frontend. Não foi afirmado roubo de token do site por esse caminho.

Verificação. Teste do FilesInterceptor real aceitou MIME image/png e preservou .html. Chrome headless em servidor loopback sintético usando o mesmo express.static recebeu text/html e executou marcador JavaScript inofensivo.

Correção recomendada. Detectar formato pelos bytes, decodificar/recodificar imagens quando aplicável e derivar extensão de um tipo realmente validado. Rejeitar HTML e formatos ativos, servir anexos em origem isolada sem credenciais e aplicar Content-Disposition/CSP adequados. Fechar o acesso público conforme A03.

Evidência — arquivo por arquivo

apps/api/src/modules/tickets/client-tickets.controller.ts:17-23

```
17: const ticketStorage = diskStorage({
18:   destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
19:   filename: (_req, file, cb) => {
20:     const unique = randomUUID();
21:     const ext = extname(file.originalname) || '.bin';
22:     cb(null, `${unique}${ext}`);
23:   },
```

apps/api/src/modules/tickets/client-tickets.controller.ts:85-93

```
85:   @UseInterceptors(FilesInterceptor('files', 10, {
86:     storage: ticketStorage,
87:     limits: { fileSize: MAX_FILE_SIZE },
88:     fileFilter: (_req, file, cb) => {
89:       if (ALLOWED_MIME.test(file.mimetype)) {
90:         cb(null, true);
91:       } else {
92:         cb(new BadRequestException(`Tipo de arquivo não permitido: ${file.mimetype}`), false);
93:       }
```

apps/api/src/app.module.ts:42-46

```
42:   ServeStaticModule.forRoot({
43:     rootPath: join(__dirname, '..', 'uploads'),
44:     serveRoot: '/uploads',
45:     serveStaticOptions: { index: false },
46:   })),
```

apps/api/src/modules/tickets/tickets.controller.ts:26-32

```
26: const ticketStorage = diskStorage({
27:   destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
28:   filename: (_req, file, cb) => {
29:     const unique = randomUUID();
30:     const ext = extname(file.originalname) || '.bin';
31:     cb(null, `${unique}${ext}`);
32:   },
```

apps/api/src/modules/maintenance/maintenance.controller.ts:24-30

```
24: const maintenanceStorage = diskStorage({
25:   destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
26:   filename: (_req, file, cb) => {
27:     const unique = randomUUID();
28:     const ext = extname(file.originalname) || '.bin';
29:     cb(null, `${unique}${ext}`);
30:   },
```

apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:22-28

```
22: const maintenanceStorage = diskStorage({
23:   destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
24:   filename: (_req, file, cb) => {
25:     const unique = randomUUID();
26:     const ext = extname(file.originalname) || '.bin';
27:     cb(null, `${unique}${ext}`);
28:   },
```

apps/api/src/modules/followups/followups.controller.ts:31-37

```
31: const followupStorage = diskStorage({
32:   destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
33:   filename: (_req, file, cb) => {
34:     const unique = randomUUID();
35:     const ext = extname(file.originalname) || '.bin';
36:     cb(null, `${unique}${ext}`);
37:   },
```

apps/api/src/modules/catalog/catalog.controller.ts:32-37

```
32: const mediaStorage = diskStorage({
33:   destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
34:   filename: (_req, file, cb) => {
35:     const ext = extname(file.originalname) || '.bin';
36:     cb(null, `${randomUUID()}${ext}`);
37:   },
```

apps/api/src/modules/inventory/inventory.controller.ts:43-48

```
43: const mediaStorage = diskStorage({
44:   destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
45:   filename: (_req, file, cb) => {
46:     const ext = extname(file.originalname) || '.bin';
47:     cb(null, `${randomUUID()}${ext}`);
48:   },
```

A11 · Template de etiqueta injeta script no HTML de impressão do frontend

ALTA

XSS e conteúdo ativo

Problema. O backend persiste layout como string sem validar sua estrutura. `parseTemplate` apenas exige `elements` como array e faz spread do objeto. `widthMm/heightMm` não são números garantidos em runtime. `openPrintWindow` interpola `heightMm` sem escape dentro de `style` via `document.write`; uma string que fecha `style` e abre `script` executa na origem do frontend.

Explorabilidade. Atacante pode gravar template via `settings.manage`. Uma vítima usa esse template e chega à impressão HTML de fallback, com popup permitido. Não é vetor anônimo. O payload de prova continha apenas um marcador em `window.opener`, sem coleta de dados ou tráfego externo.

Impacto. XSS persistente na origem do frontend. O popup herda a origem e acessa seu `opener`; scripts nessa origem podem acessar tokens guardados no `localStorage` e agir como a vítima, inclusive outra conta com mais permissões.

Verificação. Chrome headless executou as funções reais `parseTemplate` e `openPrintWindow` extraídas do código. `heightMm` malicioso permaneceu string e o marcador foi escrito no `opener`. O CSP atual permite scripts inline, portanto não bloqueia esse caminho.

Correção recomendada. Validar o JSON do layout no backend e no parser com schema estrito: dimensões finitas e limitadas, `columns` inteiro, tipos/elementos permitidos. Construir a impressão com APIs DOM e propriedades tipadas, sem interpolar valores não confiáveis em `document.write`. Reduzir scripts inline no CSP como defesa adicional.

Evidência — arquivo por arquivo

`packages/shared/src/zod/index.ts:456-459`

```
456: export const createLabelTemplateSchema = z.object({
457:   name: z.string().min(1, 'Nome é obrigatório'),
458:   layout: z.string().min(1, 'Layout é obrigatório'),
459: });
```

`apps/api/src/modules/labels/labels.controller.ts:52-63`

```
52:   @Post('templates')
53:   @RequirePermission('settings.manage')
54:   async createTemplate(@Body(new ZodValidationPipe(createLabelTemplateSchema)) body: { name: string;
    layout: string }) {
55:     const data = await this.labelsService.createTemplate(body);
56:     return { data, message: 'Template criado' };
57:   }
58:
59:   @Put('templates/:id')
60:   @RequirePermission('settings.manage')
61:   async updateTemplate(@Param('id') id: string, @Body(new
    ZodValidationPipe(updateLabelTemplateSchema)) body: { name?: string; layout?: string }) {
62:     const data = await this.labelsService.updateTemplate(id, body);
63:     return { data, message: 'Template atualizado' };
```

apps/api/src/modules/labels/labels.service.ts:170-177

```

170:     async createTemplate(data: { name: string; layout: string }) {
171:         return this.prisma.labelTemplate.create({ data });
172:     }
173:
174:     async updateTemplate(id: string, data: { name?: string; layout?: string }) {
175:         const existing = await this.prisma.labelTemplate.findUnique({ where: { id } });
176:         if (!existing) throw new NotFoundException('Template não encontrado');
177:         return this.prisma.labelTemplate.update({ where: { id }, data });

```

apps/web/src/lib/label-template.ts:168-175

```

168: export function parseTemplate(layout: string): LabelTemplate | null {
169:     try {
170:         const obj = JSON.parse(layout);
171:         if (!obj || !Array.isArray(obj.elements)) return null;
172:         return { ...blankTemplate(), ...obj, elements: obj.elements };
173:     } catch {
174:         return null;
175:     }

```

apps/web/src/app/dashboard/etiquetas/page.tsx:210-219

```

210:     const openPrintWindow = (labelsHtml: string, columns: number) => {
211:         const origin = window.location.origin;
212:         const fixedHtml = labelsHtml.replace(/src="\//brand\\/g, `src="${origin}/brand/`);
213:         const popup = window.open("", "_blank", "width=900,height=700");
214:         if (!popup) { toast.error("Habilite popups para este site para imprimir etiquetas."); return; }
215:         const labelWidth = activeTemplate.widthMm;
216:         const labelHeight = activeTemplate.heightMm;
217:         const pageW = labelWidth * columns;
218:         const qrPx = Math.max(Math.round((labelHeight - 8) * 3.78), 20);
219:         popup.document.write(`<!DOCTYPE html>

```

apps/web/src/app/dashboard/etiquetas/page.tsx:224-229

```

224: <style>
225: * { box-sizing: border-box; }
226: html, body { margin: 0; padding: 0; background: white; font-family: Arial, sans-serif; }
227: @page { size: ${pageW}mm ${labelHeight}mm; margin: 0; }
228: .sheet { display: grid; grid-template-columns: repeat(${columns}, ${labelWidth}mm); gap: 0; width:
    ${pageW}mm; }
229: .sheet > div { width: ${labelWidth}mm !important; height: ${labelHeight}mm !important; overflow:
    hidden !important; padding: 1.5mm !important; background: white !important; }

```

apps/web/src/app/dashboard/etiquetas/page.tsx:310-315

```

310:         try {
311:             printer = await getDefaultPrinter();
312:         } catch (e: any) {
313:             toast.error(`${browserPrintErrorMsg(e?.message)} Usando navegador.`);
314:             setTimeout(() => handleHtmlPrint(), 300);
315:             return;

```

apps/web/next.config.ts:14

```

14:     "script-src 'self' 'unsafe-inline' 'unsafe-eval'",

```

apps/web/src/lib/auth-context.tsx:151-154

```

151:
152:     localStorage.setItem("accessToken", res.accessToken);
153:     localStorage.setItem("refreshToken", res.refreshToken);
154:     localStorage.setItem("userType", type);

```

A12 · Listagem de clientes devolve passwordHash sem necessidade

MÉDIA

Adicional: exposição de hashes

Problema. GET /clients/users permite settings.view e retorna o resultado integral de externalUser.findMany. O interceptor de resposta não faz remoção desses campos.

Explorabilidade. Conta interna com settings.view, como o papel Gestor do seed. Não é exposição anônima nem ausência do guard; é excesso de dados na resposta para um privilégio de consulta.

Impacto. Vazamento de hashes de senha de clientes e possibilidade de tentativas offline contra senhas fracas. O hash não equivale a senha em texto puro e a auditoria não tentou quebrá-lo.

Verificação. Teste do ClientsService real confirmou consulta sem select/omit e retorno de passwordHash sintético. Este achado é adicional aos cinco temas pedidos: exposição de credencial, não segredo hardcoded.

Correção recomendada. Usar select explícito/DTO de saída com somente os campos administrativos necessários. Excluir passwordHash e quaisquer tokens/PIN de todas as respostas de cadastro e consulta; manter hashes apenas no caminho de autenticação.

Evidência — arquivo por arquivo

apps/api/src/modules/clients/clients.controller.ts:157-168

```
157:   @Get('users')
158:   @UseGuards(JwtAuthGuard, PermissionsGuard)
159:   @RequirePermission('settings.view')
160:   async findAllExternalUsers(
161:     @Query('companyId') companyId?: string,
162:     @Query('page') page?: string,
163:     @Query('limit') limit?: string,
164:   ) {
165:     const p = Math.max(1, parseInt(page ?? '1', 10) || 1);
166:     const l = Math.min(200, Math.max(1, parseInt(limit ?? '50', 10) || 50));
167:     const result = await this.clientsService.findAllExternalUsers({ companyId, skip: (p - 1) * l,
168:       take: l });
169:     return { data: result.data, total: result.total, page: p, limit: l };
```

apps/api/src/modules/clients/clients.service.ts:192-207

```
192:   async findAllExternalUsers(params?: { companyId?: string; skip?: number; take?: number }) {
193:     const where = params?.companyId ? { companyId: params.companyId } : {};
194:     const [data, total] = await Promise.all([
195:       this.prisma.externalUser.findMany({
196:         where,
197:         include: {
198:           company: { select: { id: true, name: true } },
199:           project: { select: { id: true, name: true } },
200:         },
201:         orderBy: [{ company: { name: 'asc' } }, { name: 'asc' }],
202:         skip: params?.skip ?? 0,
203:         take: params?.take ?? 200,
204:       }),
205:       this.prisma.externalUser.count({ where }),
206:     ]);
207:     return { data, total };
```

apps/api/prisma/schema.prisma:55-60

```
55: model ExternalUser {
56:   id          String   @id @default(uuid())
57:   name        String
58:   email       String   @unique
59:   passwordHash String
60:   cpf         String?  @unique // CPF do usuário externo
```

apps/api/src/interceptors/response.interceptor.ts:14-24

```
14:         map((data) => {
15:             // 204 No Content – pass through null/undefined unchanged
16:             if (data === null || data === undefined) return data;
17:             // Plain objects: spread existing keys + add success flag
18:             if (typeof data === 'object' && !Array.isArray(data)) {
19:                 return { success: true, ...data };
20:             }
21:             // Unexpected array at root: wrap it
22:             return { success: true, data };
23:         })),
24:     );
```

Recomendações priorizadas

P1 • Conter cruzamento de clientes e leitura de mídias

A01, A02, A03

Corrigir vínculo de empresa no cadastro e suspender reconciliação textual; proteger entrega de arquivos sem quebrar leitores autorizados. Revisar os vínculos já criados em procedimento aprovado e auditável.

P2 • Conter mudança de patrimônio e XSS persistente

A04, A10, A11

Autorizar o patrimônio associado, validar arquivos pelos bytes e tornar o layout de etiquetas tipado. Revisar templates/arquivos existentes com aprovação, preservando evidências.

P3 • Eliminar defaults e verificar eventual exposição operacional

A08, A09

Verificar de forma privada se defaults chegaram à produção, retirar bootstrap previsível e executar rotação planejada quando aplicável. Não publicar valores em issues ou logs.

P4 • Aplicar regras de integridade e minimizar respostas

A05, A06, A07, A12

Fechar rotas alternativas, proteger assinaturas, separar fechamento de execução e remover hashes das respostas. Adicionar testes negativos por audiência e permissão.

P5 • Sustentar a correção

Todos

Adicionar testes de autorização ao CI e secret scanning de commits/bundles. Avaliar RLS como defesa em profundidade somente com desenho compatível com a conexão Prisma; não supor que habilitar RLS substitui as políticas da API.

Cuidado operacional

Esta máquina e o Supabase atendem produção. Não executar seed, reset, migrações destrutivas, testes com dados reais ou remoção indiscriminada de mídias. Planejar correções e verificação em fixtures/banco isolado. O relatório e as issues são propostas: nenhum ticket foi publicado e nenhuma correção/deploy foi feito.

Cruzamento UI × API

Gates encontrados no código da aplicação. Exibições de nome de papel e seletores de responsáveis, sem restrição de ação, não foram confundidos com verificações de autorização. As permissões específicas de cada endpoint constam da matriz seguinte.

Administrador / hasPermission

apps/web/src/lib/auth-context.tsx:183-190

Servidor: PermissionsGuard:37-51 + JwtStrategy:26-84

Equivalente no servidor; papel reidratado do banco.

Internos / menu por screen.action

apps/web/src/components/dashboard-layout.tsx:58-62; apps/web/src/app/page.tsx:57-76

Servidor: Guards dos controllers internos; tickets/my-internal com req.user.id

Navegação não é usada como fronteira de segurança. Internos continua sem manutenção.

tickets.view, maintenance.view, inventory.view

apps/web/src/app/dashboard/page.tsx:115-125, 316

Servidor: tickets.controller:43; maintenance.controller:44,58; inventory.controller:174

Listagens/estatísticas com permissão no servidor. Filtro visual de responsável não significa tenant interno.

create/update/delete/manage_installers; InstallerCard canManage

apps/web/src/app/dashboard/agenda/page.tsx:242-244, 546, 688, 719

Servidor: schedule.controller:29-127

Todas as oito rotas têm a permissão correspondente.

access.manage

apps/web/src/app/dashboard/colaboradores/page.tsx:37; apps/web/src/app/dashboard/colaboradores/[id]/page.tsx:54

Servidor: auth.controller:89-159,219-224

Escritas exigem access.manage; consultas exigem access.view.

access.manage

apps/web/src/app/dashboard/permissoes/page.tsx:136

Servidor: access.controller:47-102

Servidor exige manage_roles/manage_permissions, mais específico que a UI.

settings.manage

apps/web/src/app/dashboard/terceirizados/page.tsx:30

Servidor: clients.controller:48-85

Equivalente nas escritas; settings.view nas consultas.

isAdmin = access.manage

apps/web/src/app/dashboard/minhas-os/page.tsx:56,74,401

Servidor: clients.controller:48-61; maintenance.controller:44-71

Nome isAdmin é impreciso; backend exige settings.view/maintenance.view. Não é ausência de controle.

Técnico/Gestor/Administrador para mídia

apps/web/src/app/dashboard/cadastros/page.tsx:122,436; apps/web/src/app/dashboard/estoque/page.tsx:208,1183

Servidor: catalog.controller:120-150; inventory.controller:61-100

Papéis também checados no handler, além do guard. Isso não valida bytes/extensão (A10).

Assumir sem dono / transferir só gestor

apps/web/src/app/dashboard/chamados/page.tsx:103,611-625

Servidor: tickets.controller:175-210; tickets.service:229-246,283-287

A05: reassign protegido, assign-with-pin contorna a restrição.

canDelete/canSign/canConfirm com isLocked

apps/web/src/components/os-forms/os-followup-section.tsx:252-256,303

Servidor: maintenance.service:427-461

Editar e confirmar têm controles; exclusão não respeita isLocked (A06).

Assinatura existente esconde nova coleta

apps/web/src/app/portal-cliente/manutencao/page.tsx:186-205

Servidor: maintenance.service:326-340

A06: API permite sobrescrever assinatura preexistente.

Cobertura completa dos handlers

Uma linha por handler. "Controle observado" descreve a política localizada, não uma garantia de ausência de outras falhas. As linhas abrangem decorators e corpo do método. IDs internos globais são governados por permissão, não pela empresa do funcionário. /uploads foi auditado separadamente em A03/A10.

apps/api/src/app.controller.ts

Health público, sem dados de negócio.

Linhas	Método e rota	Política observada	Resultado
5-8	GET /health	Público de cadastro/login/health; sem guard RBAC	Controle observado

apps/api/src/modules/access/access.controller.ts

RBAC interno global; todos os handlers têm RequirePermission para consulta/gestão de papéis e permissões.

Linhas	Método e rota	Política observada	Resultado
33-38	GET /access/roles	access.view; interno/global	Controle observado
40-45	GET /access/roles/:id	access.view; interno/global	Controle observado
47-52	POST /access/roles	access.manage_roles; interno/global	Controle observado
54-62	PUT /access/roles/:id	access.manage_roles; interno/global	Controle observado
64-69	DELETE /access/roles/:id	access.manage_roles; interno/global	Controle observado
75-80	GET /access/permissions	access.view; interno/global	Controle observado
82-87	POST /access/permissions	access.manage_permissions; interno/global	Controle observado
89-94	DELETE /access/permissions/:id	access.manage_permissions; interno/global	Controle observado
100-111	POST /access/roles/:id/permissions	access.manage_permissions; interno/global	Controle observado

apps/api/src/modules/assets/assets.controller.ts

RBAC interno global para consultas por código/SKU/ID, contagens, criação e movimentação; patrimônio não possui tenant externo por funcionário.

Linhas	Método e rota	Política observada	Resultado
25-39	GET /assets	assets.view; interno/global	Controle observado
41-53	GET /assets/summary	assets.view; interno/global	Controle observado
55-60	GET /assets/lookup/:assetCode	assets.lookup; interno/global	Controle observado
62-67	GET /assets/:id	assets.view; interno/global	Controle observado
69-74	GET /assets/:id/timeline	assets.view; interno/global	Controle observado
76-86	POST /assets/:id/events	assets.create; interno/global	Controle observado
88-101	POST /assets/bulk	assets.create; interno/global	Controle observado
103-108	POST /assets	assets.create; interno/global	Controle observado
110-115	PUT /assets/:id/status	assets.create; interno/global	Controle observado
117-122	PUT /assets/:id/location	assets.create; interno/global	Controle observado

apps/api/src/modules/auth/auth.controller.ts

Login/refresh/logout são fluxos públicos de sessão. Gestão usa RBAC; perfil/PIN usam identidade do JWT. Bootstrap/defaults tratados em A08/A09.

Linhas	Método e rota	Política observada	Resultado
40-60	POST /auth/login	Fluxo público de sessão	Controle observado
62-74	POST /auth/refresh	Fluxo público de sessão	Controle observado
76-87	POST /auth/logout	Fluxo público de sessão	Controle observado
89-106	POST /auth/users	access.manage; interno/global	Controle observado
108-115	POST /auth/setup-pin	JWT + identidade do chamador; sem ID alheio livre	Controle observado

Linhas	Método e rota	Política observada	Resultado
117-123	GET /auth/has-pin	JWT + identidade do chamador; sem ID alheio livre	Controle observado
125-131	GET /auth/users	access.view; interno/global	Controle observado
133-139	GET /auth/users/:id	access.view; interno/global	Controle observado
141-153	PUT /auth/users/:id	access.manage; interno/global	Controle observado
155-161	DELETE /auth/users/:id	access.manage; interno/global	Controle observado
163-167	GET /auth/me	JWT + identidade do chamador; sem ID alheio livre	Controle observado
169-178	GET /auth/me/profile	JWT + identidade do chamador; sem ID alheio livre	Controle observado
180-201	PUT /auth/me/profile	JWT + identidade do chamador; sem ID alheio livre	Controle observado
203-208	GET /auth/me/permissions	JWT + identidade do chamador; sem ID alheio livre	Controle observado
210-217	POST /auth/validate-pin	JWT + identidade do chamador; sem ID alheio livre	Controle observado
219-226	POST /auth/users/:id/reset-pin	access.manage; interno/global	Controle observado

apps/api/src/modules/catalog/catalog.controller.ts

RBAC interno global em todas as rotas. Upload tem restrição adicional de papel no handler; arquivos sofrem A10.

Linhas	Método e rota	Política observada	Resultado
51-61	GET /catalog/categories	catalog.view; interno/global	Controle observado
63-68	GET /catalog/categories/:id	catalog.view; interno/global	Controle observado
70-75	POST /catalog/categories	catalog.create; interno/global	Controle observado
77-82	PUT /catalog/categories/:id	catalog.update; interno/global	Controle observado
84-89	DELETE /catalog/categories/:id	catalog.delete; interno/global	Controle observado
93-105	GET /catalog/skus	catalog.view; interno/global	Controle observado

Linhas	Método e rota	Política observada	Resultado
107-112	GET /catalog/skus/:id	catalog.view; interno/global	Controle observado
114-145	POST /catalog/skus	catalog.create; interno/global	A10
147-152	PUT /catalog/skus/:id	catalog.update; interno/global	Controle observado
154-159	DELETE /catalog/skus/:id	catalog.delete; interno/global	Controle observado

apps/api/src/modules/clients/clients.controller.ts

Login e buscas de cadastro são públicos. Demais operações exigem settings.view/manage. Lista de usuários tem excesso de campos (A12). Busca pública permite descobrir empresa para A01.

Linhas	Método e rota	Política observada	Resultado
25-31	POST /clients/login	Público de cadastro/login/health; sem guard RBAC	Controle observado
34-40	GET /clients/companies/search	Público de cadastro/login/health; sem guard RBAC	Controle observado
43-49	GET /clients/companies/:companyId/projects-public	Público de cadastro/login/health; sem guard RBAC	Controle observado
52-58	GET /clients/contractors	settings.view; interno/global	Controle observado
60-66	PUT /clients/contractors/:id	settings.manage; interno/global	Controle observado
68-74	DELETE /clients/contractors/:id	settings.manage; interno/global	Controle observado
77-83	GET /clients/companies	settings.view; interno/global	Controle observado
85-91	GET /clients/companies/:id	settings.view; interno/global	Controle observado
93-99	POST /clients/companies	settings.manage; interno/global	Controle observado
101-107	PUT /clients/companies/:id	settings.manage; interno/global	Controle observado
109-115	DELETE /clients/companies/:id	settings.manage; interno/global	Controle observado
118-124	GET /clients/companies/:companyId/projects	settings.view; interno/global	Controle observado
126-135	POST /clients/companies/:companyId/projects	settings.manage; interno/global	Controle observado

Linhas	Método e rota	Política observada	Resultado
137-146	PUT /clients/projects/:id	settings.manage; interno/global	Controle observado
148-154	DELETE /clients/projects/:id	settings.manage; interno/global	Controle observado
157-169	GET /clients/users	settings.view; interno/global	A12
171-182	POST /clients/users	settings.manage; interno/global	Controle observado

apps/api/src/modules/followups/client-followups.controller.ts

Dois handlers: assertCompany, companyId na consulta/total e comparação da empresa no detalhe. Correto após estabelecimento confiável da identidade (A01).

Linhas	Método e rota	Política observada	Resultado
20-37	GET /client/followups	JWT + tipo externo + companyId do chamador	Controle observado
39-47	GET /client/followups/:id	JWT + tipo externo + companyId do chamador	Controle observado

apps/api/src/modules/followups/followups.controller.ts

CRUD/histórico/checklists/comentários internos têm RBAC e associação pai-filho; arquivo público é A03 e upload é A10.

Linhas	Método e rota	Política observada	Resultado
50-66	GET /followups	followups.view; interno/global	Controle observado
68-73	GET /followups/:id	followups.view; interno/global	Controle observado
75-86	POST /followups	followups.create; interno/global	Controle observado
88-97	PUT /followups/:id	followups.edit; interno/global	Controle observado
99-108	PUT /followups/:id/status	followups.edit; interno/global	Controle observado
110-115	DELETE /followups/:id	followups.delete; interno/global	Controle observado
119-128	POST /followups/:id/blocks	followups.edit; interno/global	Controle observado
130-140	PUT /followups/:id/blocks/:blockId	followups.edit; interno/global	Controle observado
142-151	DELETE /followups/:id/blocks/:blockId	followups.edit; interno/global	Controle observado

Linhas	Método e rota	Política observada	Resultado
155-181	POST /followups/:id/blocks/:blockId/attachments	followups.edit; interno/global	A10
183-211	GET /followups/:id/blocks/:blockId/attachments/:attId/file	Público; apenas relação dos IDs pai/filho	A03
213-226	DELETE /followups/:id/blocks/:blockId/attachments/:attId	followups.edit; interno/global	Controle observado
230-239	POST /followups/:id/blocks/:blockId/checklist	followups.edit; interno/global	Controle observado
241-251	PUT /followups/:id/blocks/:blockId/checklist:itemId	followups.edit; interno/global	Controle observado
253-262	DELETE /followups/:id/blocks/:blockId/checklist:itemId	followups.edit; interno/global	Controle observado
266-276	POST /followups/:id/blocks/:blockId/comments	followups.edit; interno/global	Controle observado
280-285	GET /followups/:id/history	followups.view; interno/global	Controle observado

apps/api/src/modules/inventory/inventory.controller.ts

RBAC interno global: listagens, estatísticas e aprovações usam screen.action. IDs de estoque não representam tenant de cliente. Upload é A10.

Linhas	Método e rota	Política observada	Resultado
61-101	POST /inventory/entry	inventory.bipar_entrada; interno/global	A10
104-120	POST /inventory/exit	inventory.bipar_saida; interno/global	Controle observado
123-128	POST /inventory/exit-batch	inventory.bipar_saida; interno/global	Controle observado
131-137	POST /inventory/approvals/:id/approve	approvals.approve; interno/global	Controle observado
140-146	POST /inventory/approvals/:id/reject	approvals.reject; interno/global	Controle observado
149-154	POST /inventory/movements/:id/reversal	inventory.historico; interno/global	Controle observado
157-163	POST /inventory/reversals/:id/approve	approvals.approve; interno/global	Controle observado
166-171	GET /inventory/approvals/pending	approvals.view; interno/global	Controle observado
174-179	GET /inventory/stats	inventory.view; interno/global	Controle observado

Linhas	Método e rota	Política observada	Resultado
182-196	GET /inventory/movements	inventory.historico; interno/global	Controle observado
199-207	GET /inventory/balances	inventory.view; interno/global	Controle observado
210-215	GET /inventory/movement-types	inventory.view; interno/global	Controle observado
217-222	POST /inventory/movement-types	settings.manage; interno/global	Controle observado
224-229	PUT /inventory/movement-types/:id	settings.manage; interno/global	Controle observado
231-236	DELETE /inventory/movement-types/:id	settings.manage; interno/global	Controle observado
239-244	GET /inventory/exit-reasons	inventory.view; interno/global	Controle observado
246-251	POST /inventory/exit-reasons	settings.manage; interno/global	Controle observado
253-258	PUT /inventory/exit-reasons/:id	settings.manage; interno/global	Controle observado
260-265	DELETE /inventory/exit-reasons/:id	settings.manage; interno/global	Controle observado

apps/api/src/modules/labels/labels.controller.ts

Impressão/histórico/dados exigem labels.print/view; templates usam settings.manage. Falta de schema do layout resulta em A11, não falta de guard.

Linhas	Método e rota	Política observada	Resultado
16-21	POST /labels/print	labels.print; interno/global	Controle observado
23-28	POST /labels/print-batch	labels.print; interno/global	Controle observado
30-35	GET /labels/history	labels.view; interno/global	Controle observado
37-42	GET /labels/data/:assetId	labels.view; interno/global	Controle observado
45-50	GET /labels/templates	labels.view; interno/global	Controle observado
52-57	POST /labels/templates	settings.manage; interno/global	A11
59-64	PUT /labels/templates/:id	settings.manage; interno/global	A11

Linhas	Método e rota	Política observada	Resultado
66-71	DELETE /labels/templates/:id	settings.manage; interno/global	Controle observado

apps/api/src/modules/locations/locations.controller.ts

Cinco handlers com RequirePermission; domínio global interno, sem tenant externo exposto.

Linhas	Método e rota	Política observada	Resultado
24-34	GET /locations	locations.view; interno/global	Controle observado
36-41	GET /locations/:id	locations.view; interno/global	Controle observado
43-48	POST /locations	locations.create; interno/global	Controle observado
50-55	PUT /locations/:id	locations.update; interno/global	Controle observado
57-62	DELETE /locations/:id	locations.delete; interno/global	Controle observado

apps/api/src/modules/maintenance/client-maintenance.controller.ts

Seis handlers autenticados usam empresa do chamador; dois handlers de arquivo são públicos (A03). Assinatura de testemunha sofre A06.

Linhas	Método e rota	Política observada	Resultado
37-54	GET /client/maintenance	JWT + tipo externo + companyId do chamador	Controle observado
56-62	GET /client/maintenance/:id	JWT + tipo externo + companyId do chamador	Controle observado
64-71	GET /client/maintenance/:id/followup-blocks	JWT + tipo externo + companyId do chamador	Controle observado
74-90	PUT /client/maintenance/:id/followup-blocks/:blockId	JWT + tipo externo + companyId do chamador	Controle observado
93-103	PUT /client/maintenance/:id/witness-signature	JWT + tipo externo + companyId do chamador	A06
105-116	POST /client/maintenance/:id/followup-blocks/:blockId/lock	JWT + tipo externo + companyId do chamador	Controle observado
119-143	GET /client/maintenance/:id/attachments/:attachmentId/file	Público; apenas relação dos IDs pai/filho	A03
145-172	GET /client/maintenance/:id/followup-blocks/:blockId/attachments/:attachmentId/file	Público; apenas relação dos IDs pai/filho	A03

apps/api/src/modules/maintenance/contractor-maintenance.controller.ts

Tipo e dono verificados nos handlers autenticados. Exceções: criação associa assetId alheio (A04), dois arquivos públicos (A03), exclusão de bloco confirmado (A06) e upload (A10).

Linhas	Método e rota	Política observada	Resultado
53-70	GET /contractor/maintenance	JWT + contractor + dono da OS (criação fixa autor)	Controle observado
72-80	GET /contractor/maintenance/:id	JWT + contractor + dono da OS (criação fixa autor)	Controle observado
82-115	POST /contractor/maintenance	JWT + contractor + dono da OS (criação fixa autor)	A04
117-135	PUT /contractor/maintenance/:id/form-data	JWT + contractor + dono da OS (criação fixa autor)	Controle observado
137-154	PUT /contractor/maintenance/:id/status	JWT + contractor + dono da OS (criação fixa autor)	Controle observado
158-207	POST /contractor/maintenance/:id/attachments	JWT + contractor + dono da OS (criação fixa autor)	A10
209-218	GET /contractor/maintenance/:id/attachments	JWT + contractor + dono da OS (criação fixa autor)	Controle observado
220-250	GET /contractor/maintenance/:id/attachments/:attachmentId/file	Público; apenas relação dos IDs pai/filho	A03
252-266	DELETE /contractor/maintenance/:id/attachments/:attachmentId	JWT + contractor + dono da OS (criação fixa autor)	Controle observado
270-277	GET /contractor/maintenance/:id/followup-blocks	JWT + contractor + dono da OS (criação fixa autor)	Controle observado
279-290	POST /contractor/maintenance/:id/followup-blocks	JWT + contractor + dono da OS (criação fixa autor)	Controle observado
292-304	PUT /contractor/maintenance/:id/followup-blocks/:blockId	JWT + contractor + dono da OS (criação fixa autor)	Controle observado
306-317	DELETE /contractor/maintenance/:id/followup-blocks/:blockId	JWT + contractor + dono da OS (criação fixa autor)	A06
319-330	POST /contractor/maintenance/:id/followup-blocks/:blockId/lock	JWT + contractor + dono da OS (criação fixa autor)	Controle observado
332-354	POST /contractor/maintenance/:id/followup-blocks/:blockId/attachments	JWT + contractor + dono da OS (criação fixa autor)	A10
356-383	GET /contractor/maintenance/:id/followup-blocks/:blockId/attachments/:attId/file	Público; apenas relação dos IDs pai/filho	A03
385-401	DELETE /contractor/maintenance/:id/followup-blocks/:blockId/attachments/:attId	JWT + contractor + dono da OS (criação fixa autor)	Controle observado

apps/api/src/modules/maintenance/maintenance.controller.ts

CRUD interno global sob RBAC. my-orders usa req.user.id; listagens/totais globais são permitidos por maintenance.view. Exceções A03/A06/A07/A10.

Linhas	Método e rota	Política observada	Resultado
44-56	GET /maintenance/my-orders	maintenance.view; interno/global	Controle observado
58-71	GET /maintenance	maintenance.view; interno/global	Controle observado
73-78	GET /maintenance/:id	maintenance.view; interno/global	Controle observado
80-88	POST /maintenance	maintenance.open; interno/global	Controle observado
90-99	PUT /maintenance/:id/status	maintenance.execute; interno/global	A07
101-110	PUT /maintenance/:id/form-data	maintenance.execute; interno/global	Controle observado
114-160	POST /maintenance/:id/attachments	[maintenance.open, maintenance.execute]; interno/global	A10
162-167	GET /maintenance/:id/attachments	maintenance.view; interno/global	Controle observado
169-199	GET /maintenance/:id/attachments/:attachmentId/file	Público; apenas relação dos IDs pai/filho	A03
201-210	DELETE /maintenance/:id/attachments/:attachmentId	maintenance.execute; interno/global	Controle observado
214-219	GET /maintenance/:id/followup-blocks	maintenance.view; interno/global	Controle observado
221-229	POST /maintenance/:id/followup-blocks	maintenance.execute; interno/global	Controle observado
231-240	PUT /maintenance/:id/followup-blocks:blockId	maintenance.execute; interno/global	Controle observado
242-250	DELETE /maintenance/:id/followup-blocks:blockId	maintenance.execute; interno/global	A06
252-260	POST /maintenance/:id/followup-blocks:blockId/lock	maintenance.execute; interno/global	Controle observado
262-283	POST /maintenance/:id/followup-blocks:blockId/attachments	maintenance.execute; interno/global	A10
285-312	GET /maintenance/:id/followup-blocks:blockId/attachments:attId/file	Público; apenas relação dos IDs pai/filho	A03
314-327	DELETE /maintenance/:id/followup-blocks:blockId/attachments:attId	maintenance.execute; interno/global	Controle observado

apps/api/src/modules/purchases/purchases.controller.ts

Cinco handlers com view/create/approve/receive no servidor; escopo operacional interno global.

Linhas	Método e rota	Política observada	Resultado
16-28	GET /purchases	purchases.view; interno/global	Controle observado
30-35	GET /purchases/:id	purchases.view; interno/global	Controle observado
37-42	POST /purchases	purchases.create; interno/global	Controle observado
44-49	PUT /purchases/:id/status	purchases.approve; interno/global	Controle observado
51-65	POST /purchases/:id/receive	purchases.receive; interno/global	Controle observado

apps/api/src/modules/registration/registration.controller.ts

Três fluxos públicos com throttling. Cadastro de cliente tem A01/A02. Cadastro/login contractor usam validação e hash; nenhum dado real foi criado.

Linhas	Método e rota	Política observada	Resultado
18-38	POST /register/client	Público de cadastro/login/health; sem guard RBAC	A01, A02
41-56	POST /register/contractor	Público de cadastro/login/health; sem guard RBAC	Controle observado
59-67	POST /register/contractor/login	Público de cadastro/login/health; sem guard RBAC	Controle observado

apps/api/src/modules/schedule/schedule.controller.ts

Oito handlers com permissões específicas no servidor, inclusive conflitos/listagem e gestão de instaladores; domínio global interno.

Linhas	Método e rota	Política observada	Resultado
29-34	GET /schedule/installers	schedule.view; interno/global	Controle observado
36-45	PUT /schedule/installers/:userId	schedule.manage_installers; interno/global	Controle observado
47-72	GET /schedule	schedule.view; interno/global	Controle observado
74-91	GET /schedule/conflicts	schedule.view; interno/global	Controle observado
93-98	GET /schedule/:id	schedule.view; interno/global	Controle observado

Linhas	Método e rota	Política observada	Resultado
100-108	POST /schedule	schedule.create; interno/global	Controle observado
110-118	PUT /schedule/:id	schedule.update; interno/global	Controle observado
120-128	DELETE /schedule/:id	schedule.delete; interno/global	Controle observado

apps/api/src/modules/suppliers/suppliers.controller.ts

Cinco handlers com permissões específicas view/create/update/delete; domínio global interno.

Linhas	Método e rota	Política observada	Resultado
24-34	GET /suppliers	suppliers.view; interno/global	Controle observado
36-41	GET /suppliers/:id	suppliers.view; interno/global	Controle observado
43-48	POST /suppliers	suppliers.create; interno/global	Controle observado
50-55	PUT /suppliers/:id	suppliers.update; interno/global	Controle observado
57-62	DELETE /suppliers/:id	suppliers.delete; interno/global	Controle observado

apps/api/src/modules/tickets/client-tickets.controller.ts

Cinco handlers vinculam dados ao externalUserId autenticado e validam posse em detalhe/anexos. A10 na validação de arquivo; entrega estática A03.

Linhas	Método e rota	Política observada	Resultado
46-51	GET /client/tickets/current	JWT + externo + externalUserId do chamador	Controle observado
53-70	GET /client/tickets	JWT + externo + externalUserId do chamador	Controle observado
72-81	GET /client/tickets/:id	JWT + externo + externalUserId do chamador	Controle observado
84-122	POST /client/tickets	JWT + externo + externalUserId do chamador	A10
125-158	POST /client/tickets/:id/attachments	JWT + externo + externalUserId do chamador	A10

apps/api/src/modules/tickets/tickets.controller.ts

Rotas operacionais têm RBAC. my-internal/mensagens/avaliação usam req.user.id e conferem autor; criação interna verifica usuário. A05 em assign-with-pin; A10 no upload.

Linhas	Método e rota	Política observada	Resultado
43-57	GET /tickets	tickets.view; interno/global	Controle observado
59-64	GET /tickets/internal-users	tickets.view; interno/global	Controle observado
66-70	GET /tickets/my-internal	JWT + req.user.id; posse no fluxo my-internal	Controle observado
72-76	GET /tickets/my-internal/pending-rating	JWT + req.user.id; posse no fluxo my-internal	Controle observado
78-113	POST /tickets/internal	JWT + req.user.id; posse no fluxo my-internal	A10
115-120	GET /tickets/:id	tickets.view; interno/global	Controle observado
122-166	POST /tickets	tickets.triage; interno/global	A10
168-173	PUT /tickets/:id/assign	tickets.assign; interno/global	Controle observado
175-186	PUT /tickets/:id/assign-with-pin	tickets.view; interno/global	A05
188-199	PUT /tickets/:id/close-with-pin	tickets.view; interno/global	Controle observado
201-212	PUT /tickets/:id/reassign	tickets.triage; interno/global	Controle observado
214-219	PUT /tickets/:id/status	tickets.triage; interno/global	Controle observado
221-226	POST /tickets/:id/messages	tickets.view; interno/global	Controle observado
228-236	POST /tickets/my-internal/:id/messages	JWT + req.user.id; posse no fluxo my-internal	Controle observado
238-246	POST /tickets/my-internal/:id/rating	JWT + req.user.id; posse no fluxo my-internal	Controle observado

Validações e rastreabilidade

Os scripts de verificação transpõem métodos do código real para dependências sintéticas. Nenhum PrismaClient real é instanciado. As duas provas de navegador usam servidor loopback temporário e Chrome headless com rede da página restrita à origem sintética. Não houve exfiltração nem acesso a credenciais.

Grupo	Evidência registrada
Offline	offline-verification.json — 13 verificações confirmadas; inclui controles negativos corretos.
Navegador	browser-verification.json — 2 provas confirmadas: template de etiquetas e HTML em express.static.
Código	source-manifest.json — SHA-256 dos arquivos usados nos trechos; findings.json conserva evidências redigidas.
Cobertura	scan-evidence.json e route-coverage.json/md — inventário completo e classificação por rota.

Referências complementares

Referência complementar para negar por padrão e conferir autorização por recurso/requisição (A01–A07).

[OWASP Authorization Cheat Sheet](#)

Referência complementar para validar conteúdo, nomes e armazenamento de uploads (A03/A10).

[OWASP File Upload Cheat Sheet](#)

Referência complementar para encoding contextual e sinks seguros (A10/A11).

[OWASP XSS Prevention Cheat Sheet](#)

Referência complementar para ciclo de vida, distribuição e rotação de segredos (A08/A09).

[OWASP Secrets Management Cheat Sheet](#)

Uso dos próximos blocos

As issues a seguir encerram o PDF. Cada bloco é completo e pode ser copiado para o GitHub; github-issues.md oferece o mesmo texto sem quebras de paginação. Agrupamentos: as sete rotas de arquivo ficam em A03; violações de assinatura em A06; credenciais de seed/docs/histórico em A08; variantes de upload em A10. Nenhuma issue foi enviada ao GitHub.

ISSUES PARA O GITHUB

Texto integral em Markdown · 12 issues acionáveis · labels sugeridas

--- ISSUE 1 ---

[Segurança] Autocadastro permite ingressar em empresa escolhida pelo solicitante
Labels sugeridas: security, alta

Identificador: A01 | Base: f3b2727190adbf96c9d35dc57b927e79f5c85275

Problema e explorabilidade

POST /register/client é público e usa companyId informado no corpo depois de verificar apenas se a empresa existe. A conta nasce ativa. Também aceita projectId sem validar a relação com a empresa. O login passa a emitir uma identidade legítima da empresa escolhida; os filtros posteriores por companyId deixam de distinguir o invasor dos clientes reais.

Cadastro público habilitado; atacante conhece o UUID da empresa ou informa nome/CNPJ que coincidem com uma empresa existente. A busca pública de empresas fornece id, nome e CNPJ. Não requer conta prévia, convite, comprovação de vínculo ou aprovação no fluxo encontrado.

Evidência

apps/api/src/modules/registration/registration.service.ts:47-54

```ts

```
47: if (data.companyId) {
48: company = await this.prisma.company.findUnique({ where: { id: data.companyId } });
49: if (!company) throw new NotFoundException('Empresa não encontrada');
50: } else if (data.companyName) {
51: company = data.companyCnpj
52: ? await this.prisma.company.findUnique({ where: { cnpj: data.companyCnpj } })
53: : null;
54: if (!company) {
```

apps/api/src/modules/registration/registration.service.ts:66-77

```ts

```
66:         const user = await this.prisma.externalUser.create({
67:             data: {
68:                 name: data.name,
69:                 email: data.email,
70:                 passwordHash,
71:                 phone: data.phone,
72:                 city: data.city,
73:                 state: data.state,
74:                 position: data.position,
75:                 companyId: company?.id || null,
76:                 projectId: data.projectId || null,
77:             },
```

Demais pontos do mesmo achado:

- apps/api/src/modules/registration/registration.controller.ts:18-21
- apps/api/prisma/schema.prisma:65-67
- apps/api/src/modules/clients/clients.service.ts:166-175

Validação: Reprodução offline executou RegistrationService.registerClient com empresa/projeto sintéticos alheios e confirmou que os identificadores foram persistidos, sem isActive=false.

Impacto

Leitura das OS e dos acompanhamentos da empresa e ações de assinatura autorizadas a seus clientes. Não foi alegado acesso a chamados pessoais de outros usuários: esses endpoints usam externalUserId.

Sugestão de correção

Vincular empresas existentes somente por convite de uso único emitido por responsável autorizado ou por aprovação interna. Resolver companyId e projectId no servidor e validar que o projeto pertence à empresa aprovada. Deixar cadastros não aprovados inativos e não executar vinculações de dados durante o cadastro público.

Critérios de aceite

- [] Cadastro sem convite não adota empresa existente nem projeto externo.
- [] Convite é autenticado, expira, é de uso único e limita empresa/projeto.
- [] Conta pendente não acessa os portais antes de aprovação.
- [] Teste de duas empresas comprova que novos usuários não obtêm OS/acompanhamentos da empresa alheia.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Autocadastro apropria OS sem empresa por correspondência parcial de nome

Labels sugeridas: security, alta

Identificador: A02 | Base: f3b2727190adb96c9d35dc57b927e79f5c85275

Problema e explorabilidade

Depois de cadastrar um cliente, o serviço busca todas as OS sem companyId e vincula à empresa recém-selecionada/criada aquelas cujo clientName contém o nome da empresa ou é contido por ele. O solicitante controla esse nome; a semelhança textual vira autorização para alterar a posse das OS.

Existem OS com companyId nulo e clientName preenchido. Um nome curto aceito pelo schema, por exemplo SA, coincide com SALA PRIVADA na reprodução sintética. Não depende de conhecer o UUID da OS nem de reivindicar uma empresa existente (A01).

Evidência

apps/api/src/modules/registration/registration.service.ts:103-114

```

```ts
103: const companyLower = company.name.toLowerCase();
104: const toLink = unlinked
105: .filter((os) => {
106: const nameLower = os.clientName!.toLowerCase().trim();
107: return companyLower.includes(nameLower) || nameLower.includes(companyLower);
108: })
109: .map((os) => os.id);
110: if (toLink.length > 0) {
111: await this.prisma.maintenanceOS.updateMany({
112: where: { id: { in: toLink } },
113: data: { companyId: company.id },
114: });
115: }
```

```

Demais pontos do mesmo achado:

- apps/api/src/modules/registration/registration.service.ts:90-101
- packages/shared/src/zod/index.ts:83-86

Validação: Teste do serviço real confirmou updateMany de uma OS sintética SALA PRIVADA para uma empresa sintética SA criada no cadastro público.

Impacto

Transferência indevida de OS legadas para empresa do atacante e posterior acesso via portal. Trata-se de escrita entre inquilinos na operação de cadastro, não apenas de uma listagem sem filtro.

Sugestão de correção

Remover a reconciliação automática do fluxo público. Executar vinculação legada em operação interna autorizada, com relação explícita e revisão por responsável. Registrar antes/depois na auditoria e preservar os dados existentes.

Critérios de aceite

- [] Cadastro público não altera companyId de nenhuma OS existente.
- [] Nomes curtos, parciais ou coincidentes não conferem posse.
- [] Reconciliação exige permissão interna, IDs aprovados e log de auditoria.
- [] Plano de revisão dos vínculos já efetuados não apaga nem reatribui dados sem aprovação.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Anexos privados acessíveis por sete rotas públicas e por /uploads
Labels sugeridas: security, alta

Identificador: A03 | Base: f3b2727190adb96c9d35dc57b927e79f5c85275

Problema e explorabilidade

Sete handlers de entrega de arquivos usam @Public(). Validam que o anexo pertence ao ID pai da URL, mas não autenticam o leitor nem verificam empresa/dono/permissão. Além disso, ServeStaticModule publica todo o diretório uploads sem JWT. O caminho Supabase também é afetado: um chamador anônimo pode obter uma nova URL assinada por meio do handler público.

Atacante possui os IDs opacos da URL ou o nome/caminho do arquivo. A auditoria não demonstrou enumeração de UUIDs. Para /uploads, deve haver arquivo no armazenamento local; para as rotas principais de manutenção, o problema existe também com Supabase Storage.

Evidência

apps/api/src/modules/maintenance/maintenance.controller.ts:169-178

```ts

```
169: @Get('/:id/attachments/:attachmentId/file')
170: @Public()
171: async serveFile(
172: @Param('id') id: string,
173: @Param('attachmentId') attachmentId: string,
174: @Res() res: Response,
175:) {
176: const attachments = await this.maintenanceService.findAttachments(id);
177: const att = attachments.find((a) => a.id === attachmentId);
178: if (!att) throw new BadRequestException('Anexo não encontrado');
179: }
```

apps/api/src/app.module.ts:42-46

```ts

```
42:   ServeStaticModule.forRoot({
43:     rootPath: join(__dirname, '..', 'uploads'),
44:     serveRoot: '/uploads',
45:     serveStaticOptions: { index: false },
46:   })),
```

Demais pontos do mesmo achado:

- apps/api/src/modules/maintenance/maintenance.controller.ts:285-297
- apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:220-229
- apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:356-368
- apps/api/src/modules/maintenance/client-maintenance.controller.ts:119-128
- apps/api/src/modules/maintenance/client-maintenance.controller.ts:145-157
- apps/api/src/modules/followups/followups.controller.ts:183-196
- apps/api/src/modules/auth/jwt-auth.guard.ts:12-18
- apps/api/src/modules/maintenance/maintenance-media-storage.service.ts:97-103

Validação: Todos os sete handlers foram lidos; JwtAuthGuard libera @Public antes da autenticação. O teste de navegador confirmou a disponibilidade de um arquivo sintético no mesmo middleware estático, sem credenciais.

Impacto

Leitura não autorizada de fotos, vídeos, assinaturas e anexos de OS, acompanhamentos e chamados. A URL externa assinada pode expirar, mas o endpoint público pode renová-la.

Sugestão de correção

Exigir autenticação e política de leitura do recurso pai antes de servir/assinar arquivos. Cobrir interno, cliente e terceirizado sem autorizar apenas pela existência dos IDs. Retirar mídias privadas da raiz estática; se necessário, emitir links temporários vinculados à autorização, sem endpoint público que os renove indefinidamente.

Critérios de aceite

- [] Leitor anônimo recebe 401/403 em todas as sete rotas e na antiga URL estática privada.
- [] Cliente de outra empresa e terceirizado não proprietário recebem 403/404.
- [] Leitores autorizados continuam recebendo mídias locais e Supabase.
- [] Links temporários expiram e não podem ser renovados anonimamente.
- [] Testes verificam anexo, bloco e objeto pai nas três audiências.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.
 --- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Terceirizado altera patrimônio alheio usando assetId ao abrir OS
 Labels sugeridas: security, alta

Identificador: A04 | Base: f3b2727190adb96c9d35dc57b927e79f5c85275

Problema e explorabilidade

A criação de OS no portal terceirizado aceita assetId do corpo. MaintenanceService verifica somente a existência do patrimônio e o associa à OS; em seguida muda seu status para EM_MANUTENCAO. Não há vínculo de autorização entre o terceirizado e esse patrimônio.

Conta de terceirizado ativa e UUID de patrimônio existente. O patrimônio não precisa estar relacionado ao solicitante. O cadastro público de terceirizados existe, mas nenhum cadastro real foi feito.

Evidência

apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:99-103
 ```ts

```
99: this.assertContractor(req);
100: const data = await this.maintenanceService.openOS({
101: assetId: body.assetId,
102: openedByContractorId: req.user.id,
103: notes: body.notes,
104: });
```

apps/api/src/modules/maintenance/maintenance.service.ts:109-124

```
```ts
109:      include: {
110:        asset: { include: { sku: { select: { skuCode: true, name: true } } } },
111:        company: { select: { id: true, name: true } },
112:        project: { select: { id: true, name: true } },
113:        openedBy: { select: { name: true } },
114:        openedByContractor: { select: { name: true } },
115:      },
116:    },
117:  );
118:
119:  // Update asset status if asset provided
120:  if (data.assetId && asset) {
121:    await this.prisma.asset.update({
122:      where: { id: data.assetId },
123:      data: { status: 'EM_MANUTENCAO' },
124:    });
125:  }
```

Demais pontos do mesmo achado:

- apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:83-87
- apps/api/src/modules/maintenance/maintenance.service.ts:59-64

Validação: Teste do serviço real com dependências sintéticas confirmou asset.update no UUID alheio e status EM_MANUTENCAO.

Impacto

Mudança não autorizada de estado operacional e associação de patrimônio de terceiros à OS do atacante. A resposta de criação/detalhe inclui o patrimônio relacionado, expondo informações internas desse recurso.

Sugestão de correção

Não aceitar assetId livre no portal terceirizado. Se o negócio exigir essa associação, consultar uma atribuição explícita autorizada ao chamador antes de criar a OS e atualizar o patrimônio, dentro de uma transação coerente.

Critérios de aceite

- [] UUID existente, mas não atribuído ao terceirizado, não cria relação nem altera o patrimônio.
- [] Objeto autorizado mantém o fluxo válido.
- [] Resposta não inclui detalhes de patrimônio sem autorização.
- [] Teste compara estado antes/depois em caso de rejeição.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] Rota de assumir chamado contorna restrição de transferência de responsável

Labels sugeridas: security, média

Identificador: A05 | Base: f3b2727190adb96c9d35dc57b927e79f5c85275

Problema e explorabilidade

A UI só mostra Assumir quando não existe responsável e reserva Mover para Administrador/Gestor. O backend restringe reassign, mas assign-with-pin aceita tickets.view e sobrescreve assignedToInternalUserId sem rejeitar um chamado já atribuído. O PIN próprio comprova identidade, não autoriza retirar o chamado de outro técnico.

Usuário interno não gestor com tickets.view e PIN válido; chamado ainda não encerrado já atribuído a outra pessoa. Não precisa conhecer o PIN do responsável atual.

Evidência

apps/api/src/modules/tickets/tickets.service.ts:229-246

```ts

```
229: async assignWithPin(ticketId: string, userId: string, roleName: string, pin: string,
 assignedToId?: string) {
230: const ticket = await this.findById(ticketId);
231: if (ticket.status === TicketStatus.CLOSED) throw new BadRequestException('Chamado encerrado
 não pode ser alterado');
232:
233: // Only admin/gestor can delegate to another user
234: const targetId = assignedToId && this.isAdminOrGestor(roleName) ? assignedToId : userId;
235:
236: const valid = await this.authService.validatePin(userId, pin);
237: if (!valid) throw new ForbiddenException('PIN inválido');
238:
239: const user = await this.prisma.internalUser.findUnique({ where: { id: targetId } });
240: if (!user) throw new NotFoundException('Usuário interno não encontrado');
241:
242: return this.prisma.ticket.update({
243: where: { id: ticketId },
244: data: {
245: assignedToInternalUserId: targetId,
246: status: TicketStatus.IN_PROGRESS,
247: },
248: });
249: }
```

Demais pontos do mesmo achado:

- apps/web/src/app/dashboard/chamados/page.tsx:611-625
- apps/api/src/modules/tickets/tickets.controller.ts:175-184
- apps/api/src/modules/tickets/tickets.service.ts:283-287

Validação: Reprodução de TicketsService.assignWithPin confirmou a substituição de um responsável sintético pelo chamador não gestor.

#### ## Impacto

Apropriação de chamados em execução, alteração de responsabilidade e possibilidade de usar o fluxo de fechamento que passa a reconhecer o atacante como responsável.

#### ## Sugestão de correção

Permitir autoatribuição apenas se o responsável estiver vazio, em atualização atômica. Qualquer troca de responsável existente deve passar pela política de transferência, inclusive por rotas alternativas. Manter validação de PIN e auditoria.

#### ## Critérios de aceite

- [ ] Autoatribuição de chamado ocupado é negada a não gestores.
- [ ] Duas requisições concorrentes não conseguem assumir o mesmo chamado.
- [ ] Transferência exige o privilégio apropriado em todos os endpoints.
- [ ] Chamado disponível ainda pode ser assumido com PIN válido.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

# [Segurança] Bloqueios de assinatura podem ser contornados por exclusão ou substituição  
Labels sugeridas: security, média

Identificador: A06 | Base: f3b2727190adb96c9d35dc57b927e79f5c85275

#### ## Problema e explorabilidade

A UI impede excluir bloco isLocked e deixa de oferecer nova assinatura quando witnessSignature já existe. A API rejeita editar um bloco bloqueado, mas removeFollowupBlock o exclui sem verificar isLocked. clientSignWitness verifica empresa e status, porém sobrescreve uma assinatura de testemunha preexistente.

Exclusão: colaborador com maintenance.execute ou terceirizado dono da OS. Substituição: cliente da empresa da OS, ainda não encerrada. Não é acesso a objeto alheio; é bypass das regras de imutabilidade em objetos acessíveis ao chamador.

#### ## Evidência

apps/api/src/modules/maintenance/maintenance.service.ts:456-461

```ts

```
456:   async removeFollowupBlock(osId: string, blockId: string) {
457:     const block = await this.prisma.maintenanceOSFollowupBlock.findFirst({
458:       where: { id: blockId, maintenanceOSId: osId },
459:     });
460:     if (!block) throw new NotFoundException('Bloco não encontrado');
461:     await this.prisma.maintenanceOSFollowupBlock.delete({ where: { id: blockId } });
462:   }
```

apps/api/src/modules/maintenance/maintenance.service.ts:326-340

```ts

```
326: async clientSignWitness(osId: string, companyId: string, signature: string) {
327: if (!signature || !signature.startsWith('data:image/')) {
328: throw new BadRequestException('Assinatura inválida');
329: }
330: const os = await this.findById(osId);
331: if (os.companyId !== companyId) throw new ForbiddenException('OS não pertence à sua
 empresa');
332: if (os.status === MaintenanceStatus.CLOSED) throw new BadRequestException('OS já
 encerrada');
333:
334: const existingFormData = (os.formData as Record<string, unknown>) || {};
335: const newFormData = { ...existingFormData, witnessSignature: signature };
336: validateFormData(newFormData);
337:
338: return this.prisma.maintenanceOS.update({
339: where: { id: osId },
340: data: { formData: newFormData as Prisma.InputJsonValue },
341: });
342: }
```

Demais pontos do mesmo achado:

- apps/web/src/components/os-forms/os-followup-section.tsx:252-256
- apps/web/src/app/portal-cliente/manutencao/page.tsx:186-195
- apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:306-316
- apps/api/src/modules/maintenance/client-maintenance.controller.ts:93-102

- apps/api/src/modules/maintenance/maintenance.service.ts:427-433

Validação: Dois testes do serviço real confirmaram exclusão de bloco SIGNATURE isLocked=true e sobrescrita de witnessSignature. Um teste de controle confirmou que a edição comum do bloco bloqueado é rejeitada.

#### ## Impacto

Perda ou troca de evidência de aceite, comprometendo integridade e rastreabilidade das assinaturas confirmadas.

#### ## Sugestão de correção

Centralizar a política de imutabilidade e aplicá-la a edição, exclusão, anexos e assinatura. Impedir sobrescrita concorrente de assinatura existente. Correções legítimas devem gerar revisão auditável, sem destruir o aceite original.

#### ## Critérios de aceite

- [ ] Bloco confirmado não pode ser excluído ou alterado por nenhuma audiência.
- [ ] Segunda assinatura não sobrescreve witnessSignature existente.
- [ ] Duas assinaturas concorrentes preservam um único aceite original.
- [ ] Revisões autorizadas preservam histórico e registram autor/motivo.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

# [Segurança] Encerrar OS ignora a permissão específica maintenance.close  
Labels sugeridas: security, média

Identificador: A07 | Base: f3b2727190adb96c9d35dc57b927e79f5c85275

#### ## Problema e explorabilidade

O contrato de permissões define maintenance.execute e maintenance.close separadamente, mas PUT /maintenance/:id/status exige somente execute. O schema aceita CLOSED e o serviço encerra a OS sem consultar close. A revogação de close não restringe essa operação.

Papel personalizado com maintenance.execute e sem maintenance.close. Os papéis operacionais padrão do seed concedem as duas em conjunto; a separação só se manifesta quando as permissões são configuradas de forma diferente. O portal terceirizado tem política própria e não foi considerado sujeito ao RBAC interno.

#### ## Evidência

apps/api/src/modules/maintenance/maintenance.controller.ts:90-98

```
```ts
90:   @Put('/:id/status')
91:   @RequirePermission('maintenance.execute')
92:   async updateStatus(
93:     @Param('id') id: string,
94:     @Request() req: any,
95:     @Body(new ZodValidationPipe(updateMaintenanceStatusSchema)) body: { status: string; notes?:
      string },
96:   ) {
97:     const data = await this.maintenanceService.updateStatus(id, body.status, req.user.id,
      body.notes);
98:     return { data, message: 'OS atualizada' };
99:   }
```
```

apps/api/src/modules/maintenance/maintenance.service.ts:225-234

```
```ts
225:     const prevTag = os.notes?.match(/\[previousStatus:\w+\]/);
226:     updateData.notes = prevTag ? `${prevTag[0]} ${notes}` : notes;
227:   }
228:   if (status === MaintenanceStatus.CLOSED) {
229:     updateData.closedById = userId;
230:     updateData.completedAt = new Date();
231:     // Restore previous asset status from the notes field (only if asset exists)
232:     if (os.assetId) {
```

```

233:         const previousStatusMatch = os.notes?.match(/\[previousStatus:(\w+)\]/);
234:         const previousStatus = previousStatusMatch ? previousStatusMatch[1] : 'ATIVO';
235:     }

```

Demais pontos do mesmo achado:

- apps/api/prisma/seed.ts:87-90
- packages/shared/src/zod/index.ts:424-426

Validação: Teste isolado autorizou somente maintenance.execute no guard e confirmou que MaintenanceService.updateStatus ainda persiste CLOSED.

Impacto

Fechamento de OS e efeitos operacionais associados por usuário sem o privilégio específico de encerramento.

Sugestão de correção

Exigir maintenance.close quando o estado desejado for CLOSED, mantendo a política adequada às outras transições. Fazer a decisão no servidor e testar combinações de permissões, não apenas os papéis padrão.

Critérios de aceite

- [] Usuário com execute, mas sem close, não encerra OS.
- [] Revogar close tem efeito na requisição seguinte.
- [] Transições não finais continuam respeitando execute.
- [] Matriz de testes inclui papéis personalizados e as três audiências.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

[Segurança] Seed e documentação publicam credenciais fixas de administrador
Labels sugeridas: security, alta

Identificador: A08 | Base: f3b2727190adb9f6c9d35dc57b927e79f5c85275

Problema e explorabilidade

O seed calcula hashes de senha e PIN literais conhecidos, cria o usuário administrador ativo e não exige substituição no bootstrap. A documentação divulga as mesmas credenciais. Hashing em repouso não protege uma senha cujo valor inicial é público. update: {} conserva uma conta preexistente, mas a criação continua insegura.

Seed executado quando a conta padrão não existe e credenciais não trocadas depois. Não foi testado login nem verificado se o administrador de produção conserva esses valores. O risco é de bootstrap/configuração, não uma afirmação de invasão atual.

Evidência

apps/api/prisma/seed.ts:218-233

```ts

```

218: const adminPasswordHash = await bcrypt.hash('[CREDENCIAL_PADRAO_REDIGIDA]', 10);
219: const adminPinHash = await bcrypt.hash('[CREDENCIAL_PADRAO_REDIGIDA]', 10);
220:
221: const adminUser = await prisma.internalUser.upsert({
222: where: { email: 'admin@zyllen.com' },
223: update: {},
224: create: {
225: name: 'Administrador',
226: email: 'admin@zyllen.com',
227: passwordHash: adminPasswordHash,
228: pin4Hash: adminPinHash,
229: roleId: adminRole.id,
230: isActive: true,
231: },
232: });
233: console.log(' ✔ Admin user created:', adminUser.email, '(PIN: [PIN_REDIGIDO])');
234: }
235: }

```

Demais pontos do mesmo achado:

- PROJETO.md:887

Validação: Literais encontrados no código atual e no histórico desde o commit ae56108d26d255f6ed35746eed5d7776707cbeef (10/02/2026), seed.ts:130-131. Valores de senha/PIN foram redigidos neste relatório para não ampliar sua circulação.

### ## Impacto

Acesso administrativo integral às funções protegidas pelo bypass Administrador, se a conta ainda aceitar as credenciais padrão.

### ## Sugestão de correção

Remover senha/PIN padrão do seed, logs e documentação. Exigir bootstrap explícito com segredo aleatório de uso único e troca obrigatória. Verificar e rotacionar credenciais eventualmente derivadas desses valores; tratar o histórico como exposição permanente, sem depender de apagar commits.

### ## Critérios de aceite

- [ ] Sem segredo de bootstrap explícito, seed não cria administrador com credencial previsível.
- [ ] Nenhum log/documento publica senha ou PIN reais.
- [ ] Primeiro acesso exige troca de senha e criação de PIN.
- [ ] Responsável verifica a conta existente e registra a rotação sem divulgar valores.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 8 ---

--- ISSUE 9 ---

# [Segurança] Docker fornece segredo JWT público e startup aceita o valor padrão

Labels sugeridas: security, alta

Identificador: A09 | Base: f3b2727190adb96c9d35dc57b927e79f5c85275

### ## Problema e explorabilidade

docker-compose.yml define JWT\_SECRET com fallback público. O módulo JWT e a estratégia usam getOrThrow, que rejeita ausência, mas não rejeita um valor conhecido/fraco presente. ConfigModule não contém validador que bloqueie esse default.

Deployment recebe o fallback do Compose ou copia esse literal para sua configuração. O Compose atual também contém uma DATABASE\_URL SQLite desatualizada: não se afirma que ele sobe como está; a exploração depende de corrigir a conexão PostgreSQL e conservar o JWT inseguro. A configuração real da produção não foi inspecionada.

### ## Evidência

docker-compose.yml:11-14

```ts

```
11:     - DATABASE_URL=file:./dev.db
12:     - JWT_SECRET=${JWT_SECRET:-change-me-in-production}
13:     - API_PORT=3001
14:     - NODE_ENV=production
```
```

apps/api/src/modules/auth/auth.module.ts:17-20

```ts

```
17:         useFactory: (cfg: ConfigService) => ({
18:             secret: cfg.getOrThrow<string>('JWT_SECRET'),
19:             signOptions: { expiresIn: '1d' },
20:         }),
```
```

Demais pontos do mesmo achado:

- apps/api/src/modules/auth/jwt.strategy.ts:18-23
- apps/api/src/app.module.ts:26-29

Validação: Fallback atual em docker-compose.yml:12. Histórico também contém fallback no código JWT, por exemplo blob 60b0cecf4d99db928ce8394a8542900a821940d1, auth.module.ts:14. O código atual removeu esse fallback local, mas o Compose e a falta de validação permanecem.

**## Impacto**

Conhecer a chave permite forjar JWTs para IDs de usuários ativos conhecidos. A estratégia reconsulta usuários no banco, portanto inventar um ID ou apenas escrever role=Administrador no token não basta; ainda assim, a assinatura deixa de provar autenticidade.

**## Sugestão de correção**

Exigir JWT\_SECRET explicitamente no deployment e validar no startup comprimento/qualidade mínima e uma denylist dos defaults históricos. Rotacionar qualquer chave implantada derivada do literal e invalidar tokens correspondentes de forma planejada.

**## Critérios de aceite**

- [ ] Compose falha de forma clara se JWT\_SECRET não estiver definido.
- [ ] Startup rejeita o valor histórico e demais placeholders públicos.
- [ ] Configuração válida de alta entropia permite iniciar.
- [ ] Há procedimento seguro de rotação e invalidação de sessões afetadas.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 9 ---

--- ISSUE 10 ---

# [Segurança] Upload de HTML disfarçado de imagem executa JavaScript na origem da API  
Labels sugeridas: security, média

Identificador: A10 | Base: f3b2727190adb96c9d35dc57b927e79f5c85275

**## Problema e explorabilidade**

Uploads locais confiam no MIME informado no multipart e preservam a extensão original do nome. Um arquivo .html com Content-Type image/png passa pelo fileFilter; o servidor estático determina text/html pela extensão. Ao abrir o link diretamente, o navegador executa o HTML persistido.

Usuário autenticado com acesso a um fluxo de upload, inclusive cliente em chamado próprio; vítima abre o arquivo como documento. Confirmado no fluxo local de chamados, independentemente de o storage principal de OS usar Supabase. Não executa simplesmente ao aparecer dentro de uma tag img.

**## Evidência**

apps/api/src/modules/tickets/client-tickets.controller.ts:17-23  
```ts

```
17: const ticketStorage = diskStorage({
18:   destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
19:   filename: (_req, file, cb) => {
20:     const unique = randomUUID();
21:     const ext = extname(file.originalname) || '.bin';
22:     cb(null, `${unique}${ext}`);
23:   },
  ...
```

apps/api/src/modules/tickets/client-tickets.controller.ts:85-93
```ts

```
85: @UseInterceptors(FilesInterceptor('files', 10, {
86: storage: ticketStorage,
87: limits: { fileSize: MAX_FILE_SIZE },
88: fileFilter: (_req, file, cb) => {
89: if (ALLOWED_MIME.test(file.mimetype)) {
90: cb(null, true);
91: } else {
92: cb(new BadRequestException(`Tipo de arquivo não permitido: ${file.mimetype}`),
93: false);
94: }
95: }
 ...
```

apps/api/src/app.module.ts:42-46  
```ts

```
42:   ServeStaticModule.forRoot({
43:     rootPath: join(__dirname, '..', 'uploads'),
44:     serveRoot: '/uploads',
45:     serveStaticOptions: { index: false },
  ...
```

```
46:      })),
...

```

Demais pontos do mesmo achado:

- apps/api/src/modules/tickets/tickets.controller.ts:26-32
- apps/api/src/modules/maintenance/maintenance.controller.ts:24-30
- apps/api/src/modules/maintenance/contractor-maintenance.controller.ts:22-28
- apps/api/src/modules/followups/followups.controller.ts:31-37
- apps/api/src/modules/catalog/catalog.controller.ts:32-37
- apps/api/src/modules/inventory/inventory.controller.ts:43-48

Validação: Teste do FilesInterceptor real aceitou MIME image/png e preservou .html. Chrome headless em servidor loopback sintético usando o mesmo express.static recebeu text/html e executou marcador JavaScript inofensivo.

Impacto

XSS persistente na origem da API e conteúdo enganoso servido pelo domínio confiável. A API e o frontend estão em origens distintas: esse vetor, sozinho, não lê o localStorage do frontend. Não foi afirmado roubo de token do site por esse caminho.

Sugestão de correção

Detectar formato pelos bytes, decodificar/recodificar imagens quando aplicável e derivar extensão de um tipo realmente validado. Rejeitar HTML e formatos ativos, servir anexos em origem isolada sem credenciais e aplicar Content-Disposition/CSP adequados. Fechar o acesso público conforme A03.

Critérios de aceite

- [] Arquivo HTML declarado image/png é rejeitado e não fica publicamente armazenado.
- [] Extensão final é derivada do formato verificado, nunca do nome original.
- [] Arquivo rejeitado é removido e não deixa registro/arquivo órfão.
- [] Abertura direta de anexos não executa conteúdo ativo na origem autenticada.
- [] Testes cobrem todos os controladores que compartilham o padrão.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 10 ---

--- ISSUE 11 ---

[Segurança] Template de etiqueta injeta script no HTML de impressão do frontend
Labels sugeridas: security, alta

Identificador: A11 | Base: f3b2727190adbf96c9d35dc57b927e79f5c85275

Problema e explorabilidade

O backend persiste layout como string sem validar sua estrutura. parseTemplate apenas exige elements como array e faz spread do objeto. widthMm/heightMm não são números garantidos em runtime. openPrintWindow interpola heightMm sem escape dentro de style via document.write; uma string que fecha style e abre script executa na origem do frontend.

Atacante pode gravar template via settings.manage. Uma vítima usa esse template e chega à impressão HTML de fallback, com popup permitido. Não é vetor anônimo. O payload de prova continha apenas um marcador em window.opener, sem coleta de dados ou tráfego externo.

Evidência

```
apps/web/src/lib/label-template.ts:168-175
...ts
168: export function parseTemplate(layout: string): LabelTemplate | null {
169:     try {
170:         const obj = JSON.parse(layout);
171:         if (!obj || !Array.isArray(obj.elements)) return null;
172:         return { ...blankTemplate(), ...obj, elements: obj.elements };
173:     } catch {
174:         return null;
175:     }
...

```

```
apps/web/src/app/dashboard/etiquetas/page.tsx:224-229
...ts

```

```

224: <style>
225: * { box-sizing: border-box; }
226: html, body { margin: 0; padding: 0; background: white; font-family: Arial, sans-serif; }
227: @page { size: ${pageW}mm ${labelHeight}mm; margin: 0; }
228: .sheet { display: grid; grid-template-columns: repeat(${columns}, ${labelWidth}mm); gap: 0; width:
    ${pageW}mm; }
229: .sheet > div { width: ${labelWidth}mm !important; height: ${labelHeight}mm !important; overflow:
    hidden !important; padding: 1.5mm !important; background: white !important; }
...

```

Demais pontos do mesmo achado:

- packages/shared/src/zod/index.ts:456-459
- apps/api/src/modules/labels/labels.controller.ts:52-63
- apps/api/src/modules/labels/labels.service.ts:170-177
- apps/web/src/app/dashboard/etiquetas/page.tsx:210-219
- apps/web/src/app/dashboard/etiquetas/page.tsx:310-315
- apps/web/next.config.ts:14
- apps/web/src/lib/auth-context.tsx:151-154

Validação: Chrome headless executou as funções reais `parseTemplate` e `openPrintWindow` extraídas do código. `heightMm` malicioso permaneceu string e o marcador foi escrito no `opener`. O CSP atual permite `scripts inline`, portanto não bloqueia esse caminho.

Impacto

XSS persistente na origem do frontend. O popup herda a origem e acessa seu `opener`; `scripts` nessa origem podem acessar tokens guardados no `localStorage` e agir como a vítima, inclusive outra conta com mais permissões.

Sugestão de correção

Validar o JSON do layout no backend e no parser com schema estrito: dimensões finitas e limitadas, `columns` inteiro, tipos/elementos permitidos. Construir a impressão com APIs DOM e propriedades tipadas, sem interpolar valores não confiáveis em `document.write`. Reduzir `scripts inline` no CSP como defesa adicional.

Critérios de aceite

- [] POST/PUT de template rejeita dimensões string, NaN, infinitas e valores fora dos limites.
- [] Template legado inválido é recusado de forma segura, sem quebrar impressão válida.
- [] Payload que fecha `style` não executa no popup nem no `opener`.
- [] Teste de impressão cobre fallback do Browser Print e configurações persistidas.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 11 ---

--- ISSUE 12 ---

[Segurança] Listagem de clientes devolve `passwordHash` sem necessidade
Labels sugeridas: security, média

Identificador: A12 | Base: f3b2727190adb96c9d35dc57b927e79f5c85275

Problema e explorabilidade

GET `/clients/users` permite `settings.view` e retorna o resultado integral de `externalUser.findMany`.
inclui seleciona as relações, mas não exclui campos escalares do modelo, como `passwordHash`. O
interceptor de resposta não faz remoção desses campos.

Conta interna com `settings.view`, como o papel Gestor do seed. Não é exposição anônima nem ausência do
guard; é excesso de dados na resposta para um privilégio de consulta.

Evidência

apps/api/src/modules/clients/clients.service.ts:192-207

```

```ts

```

```

192: async findAllExternalUsers(params?: { companyId?: string; skip?: number; take?: number }) {
193: const where = params?.companyId ? { companyId: params.companyId } : {};
194: const [data, total] = await Promise.all([
195: this.prisma.externalUser.findMany({
196: where,
197: include: {

```

```

198: company: { select: { id: true, name: true } },
199: project: { select: { id: true, name: true } },
200: },
201: orderBy: [{ company: { name: 'asc' } }, { name: 'asc' }],
202: skip: params?.skip ?? 0,
203: take: params?.take ?? 200,
204: })),
205: this.prisma.externalUser.count({ where })),
206: });
207: return { data, total };
```

```

Demais pontos do mesmo achado:

- apps/api/src/modules/clients/clients.controller.ts:157-168
- apps/api/prisma/schema.prisma:55-60
- apps/api/src/interceptors/response.interceptor.ts:14-24

Validação: Teste do ClientsService real confirmou consulta sem select/omit e retorno de passwordHash sintético. Este achado é adicional aos cinco temas pedidos: exposição de credencial, não segredo hardcoded.

Impacto

Vazamento de hashes de senha de clientes e possibilidade de tentativas offline contra senhas fracas. O hash não equivale a senha em texto puro e a auditoria não tentou quebrá-lo.

Sugestão de correção

Usar select explícito/DTO de saída com somente os campos administrativos necessários. Excluir passwordHash e quaisquer tokens/PIN de todas as respostas de cadastro e consulta; manter hashes apenas no caminho de autenticação.

Critérios de aceite

- [] GET /clients/users não contém passwordHash em nenhum nível.
- [] Contrato de resposta é allowlist de campos necessários.
- [] Teste automatizado bloqueia vazamento de campos de credenciais.
- [] Listagens e detalhes de usuários internos/externos/terceirizados são verificados para a mesma classe de erro.

Nota: não executar testes em dados reais nem publicar segredos. Usar fixtures e banco isolado.

--- FIM ISSUE 12 ---